

Algorithm for Dynamic Modelling of Metabolism with a Multiphase Objective Function



Thomas Munn 2300490

School of Mathematics
University of Birmingham
2025

Supervised by Dr David Tourigny

Abstract

Metabolism in microbial cultures can be modelled using a system of ODEs, which is often very large and impossible to solve directly. We can reduce the problem under a steady state assumption to model a constant environment or a quasi-steady state assumption to account for a dynamic environment. This does not, however, account for how a microorganism might adapt its survival strategy in different environments. The focus of this paper is an efficient algorithm to solving a model of a microorganisms metabolism, when it has the capability to change strategy in response to environmental changes.

Contents

1	Introduction	2
1.1	Kinetic Models for Microbial Metabolism	2
1.2	Flux Balance Analysis	6
1.3	Dynamic Flux Balance Analysis	8
1.4	Report Summary	10
2	Mathematical Theory of Standard DFBA Problems	12
2.1	Required LP Theory	12
2.2	Formulation of the Standard dFBA Problem	17
2.3	Direct Method for Solving Standard dFBA Problems	19
2.4	Optimal Basis Algorithm for Solving Standard dFBA Problems	21
3	dFBA Problems with Dynamic Objective Functions	30
3.1	Formulation of Extended dFBA Problem	30
3.2	Extended Optimal Basis Algorithm	32
3.3	Demonstration of Extended Optimal Basis Algorithm on Toy Models	35
4	Biological Example of an Extended dFBA Problem	43
4.1	Description of Real World Example	43
4.2	Simulation of Real World Example	44
5	Conclusions	49
A	Further LP Theory	53
A.1	Transforming LP Problems	53
A.2	Full Rank	54
A.3	Dual Problems	55
B	Extra Functions without Analytic Expressions	56
C	Norm Definitions	58

Chapter 1

Introduction

Cultures of microorganisms are vital in the production of many important resources both in industry and in healthcare. Certain medical compounds (e.g. insulin, vaccines) and biofuels are now produced by microorganisms, as are different alcohol or dairy-based products that require a fermentation process. Therefore being able to improve the efficiency of these processes will be highly impactful to society.

Throughout this paper, we will present many examples of algorithms. The code used to generate these diagrams will be stored in a repository (Munn 2025).

1.1 Kinetic Models for Microbial Metabolism

Metabolism is the combination of chemical reactions that occur within a living being in order for that being to survive and reproduce. Metabolism can generally be divided into two processes: catabolism and anabolism. Catabolism involves breaking down ‘fuel’ chemical components (called metabolites) such as glucose into smaller components. Anabolism is then the building of larger components (e.g. proteins, nucleic acids, lipids) using these smaller parts.

We will begin by presenting the most direct approach to modelling metabolism, through a motivating example (Waldherr, Oyarzún, and Bockmayr 2014). Consider a simple organism, which is able to convert glucose (a common source of energy and carbon) into ethanol (a common by-product of respiration) and biomass, to allow the organism to grow. It is also able to convert ethanol back into pyruvate. A diagram of this organism is shown in Figure 1.1. Here ‘biomass’ refers to the average combination of components that makes up a cell, and we will not consider the specifics of building the macromolecule (large molecules that make up biomass) precursors that are required. Each of our components in this system are known as metabolites,

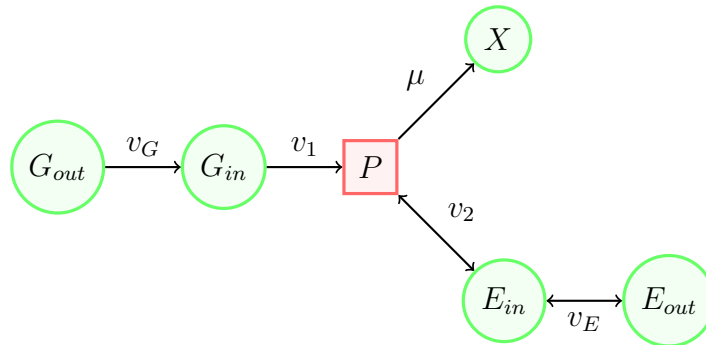


Figure 1.1: Diagram of our example metabolic network. Glucose can be transported into the cell, and is used to produce pyruvate. Pyruvate can be used to produce biomass or ethanol. Ethanol can be transported in or out of the cell, and can also be converted back into pyruvate.

since they are elements of metabolism and we can characterise them in the following way. G_{out} and E_{out} refer to the glucose and ethanol outside of the cell (extracellular) respectively. Then, G_{in} and E_{in} represent the intracellular glucose and ethanol, with pyruvate, P , also being intracellular. Since cellular organisms have outer membranes to hold the structure of the cell together, it requires work to transfer metabolites into the cell. So, we will refer to this transfer as an exchange reaction. v_G and v_E are the fluxes through the exchange reactions for glucose and ethanol respectively. We will characterise biomass, X , as a macromolecule. In general, cellular biomass itself is made up of different macromolecules (proteins, DNA, RNA, etc.), but in this simple model we represent all of this using a single variable, X .

We will construct a system of Ordinary Differential Equations (ODEs) describing how the quantities (currently defined in molar amounts) of our metabolites change over time. The quantities of our metabolites will increase depending on the reactions producing them, and decrease depending on the reactions consuming them. For example, G_{in} is produced by the glucose exchange reaction and is consumed when converted into pyruvate. This means the rate of change is proportional to $v_G - v_1$. This rate will also be proportional to the amount of biomass present, since the rate of reaction is proportional to the availability of material to catalyse it. So, as an example we will have $\dot{G}_{in} = (v_G - v_1)X$ and the rest of our equations will be derived similarly, except when biomass production is involved, as described below.

As a consequence of the way that we have constructed the biomass reaction as a series of precursors, we will say that to produce one unit of biomass we will require a large constant α units of pyruvate, so $\alpha P \rightarrow X$. We will also assume that for larger stoichiometric coefficients α (i.e. there are more complicated or a larger number of precursors required), the biomass reaction will be slower. To do this, we will scale it by a small dimensionless factor ϵ and we will take the product $\alpha\epsilon$ as a constant. Putting all of this together gives us the following system of ODEs:

$$\begin{aligned}\dot{G}_{out} &= -v_G X, \\ \dot{E}_{out} &= v_E X, \\ \dot{G}_{in} &= (v_G - v_1)X, \\ \dot{P} &= (v_1 - v_2 - \alpha\epsilon\mu)X, \\ \dot{E}_{in} &= (v_2 - v_E)X, \\ \dot{X} &= \epsilon\mu X.\end{aligned}\tag{1.1}$$

Denoting our intracellular metabolites, extracellular metabolites and fluxes in vector notation, we have

$$\mathbf{u} = \begin{pmatrix} G_{in} \\ P \\ E_{in} \end{pmatrix}, \mathbf{w} = \begin{pmatrix} G_{out} \\ E_{out} \end{pmatrix}, \mathbf{v}_u = \begin{pmatrix} v_1 \\ v_2 \end{pmatrix}, \mathbf{v}_w = \begin{pmatrix} v_G \\ v_E \end{pmatrix}, \text{ and } v_X = \mu\tag{1.2}$$

so we can also write our system in the following form:

$$\begin{aligned}\dot{\mathbf{w}} &= -S_w^w \mathbf{v}_w X, \\ \dot{\mathbf{u}} &= (S_w^u \mathbf{v}_w + S_u^u \mathbf{v}_u - \alpha\epsilon S_X^u \mu)X, \\ \dot{X} &= \epsilon S_X^X \mu X,\end{aligned}\tag{1.3}$$

where the stoichiometric matrices are given by

$$S_w^w = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}, S_w^u = \begin{pmatrix} 1 & 0 \\ 0 & 0 \\ 0 & -1 \end{pmatrix}, S_u^u = \begin{pmatrix} -1 & 0 \\ 1 & -1 \\ 0 & 1 \end{pmatrix}, S_X^u = \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix}, S_X^X = 1.\tag{1.4}$$

This representation is helpful, since it allows us to visualise what a system with more variables might look like, by increasing the sizes of our vectors and matrices. In our network representation, this would be the equivalent of more nodes and arcs between them. In fact, we can represent a metabolic system with any level of complexity in this same way.

Currently, each of these variables are in molar amounts instead of concentrations. To convert these into concentrations, we need to know more about the volumes within the system. For our simple metabolic network, we will assume that the volume of the extracellular environment θ_e is constant, however it need not be in general. The volume of the cell is dependent on the amount of macromolecules. We can write this as a function $\theta_c(t, X)$. Then we can write our concentrations as

$$\mathbf{x} = \frac{\mathbf{w}}{\theta_e}, \mathbf{y} = \begin{pmatrix} g \\ p \\ e \end{pmatrix} = \frac{\mathbf{u}}{\theta_c(t, X)}. \quad (1.5)$$

To simplify our example even further, we can take $\theta_e = 1$ and $\theta_c(t, X) = X$. This then means that $\mathbf{x} = \mathbf{w}$, so $\dot{\mathbf{x}} = \dot{\mathbf{w}}$. We will use the notation $p = P/X$ as the intracellular concentration of pyruvate, so we will have

$$\begin{aligned} X\dot{p} &= \dot{P} - p\dot{X} = (v_1 - v_2 - \alpha\epsilon\mu)X - \epsilon\mu pX, \\ \implies \dot{p} &= v_1 - v_2 - \alpha\epsilon\mu - \epsilon\mu p. \end{aligned} \quad (1.6)$$

Similarly, for intracellular glucose ($g = G_{in}/X$) and ethanol ($e = E_{in}/X$) concentrations we have

$$\dot{g} = v_G - v_1 - g\epsilon\mu, \quad (1.7)$$

$$\dot{e} = v_2 - v_E - e\epsilon\mu. \quad (1.8)$$

We can now write our system again, but in terms of concentrations instead of molar amounts:

$$\begin{aligned} \dot{\mathbf{x}} &= -S_w^w \mathbf{v}_w X, \\ \dot{\mathbf{y}} &= S_w^u \mathbf{v}_w + S_u^u \mathbf{v}_u - \alpha\epsilon S_X^u \mu - \epsilon\mu \mathbf{y}, \\ \dot{X} &= \epsilon S_X^X \mu X, \end{aligned} \quad (1.9)$$

where the stoichiometry matrices are the same as before. The matrix S_w^w shows how the organism can transfer extracellular metabolites into the cell. The matrix S_u^u contains all of the intracellular reactions and the matrix S_X^u holds the information for building the precursors required for biomass.

Again, this notation is helpful as we can represent any metabolic system in this way. Our organisms could be much more complicated than the simple one presented in Figure 1.1. As an example, we will show the metabolic network for central carbon metabolism for a species of *Escherichia coli* (*E. Coli*) in Figure 1.2. In this diagram, each arrow represents a reaction, purple boxes represent external metabolites and orange boxes represent internal metabolites. We can use this representation to identify components of metabolism. For example, the circle of reactions in the bottom right of the diagram is the Krebs cycle (or TCA cycle).

Our next step is to build up a more generic model, which can be applied to any combination of extracellular metabolites and species of microorganism. We will start with a network similar to the one presented in Figure 1.2.

Suppose our network has m intracellular metabolites and n reactions. To start building a mathematical representation of our metabolic network, we will create a stoichiometry matrix S with m rows and n columns. Each row represents a metabolite and each column represents a reaction. The element in the i th row and j th column, S_{ij} , represents the coefficient of the i th metabolite in the j th reaction. If we consider $\mathbf{v}$, the vector of fluxes through each of our reactions and $\mathbf{y}$, the vector of concentrations of intracellular metabolites; then the multiplication

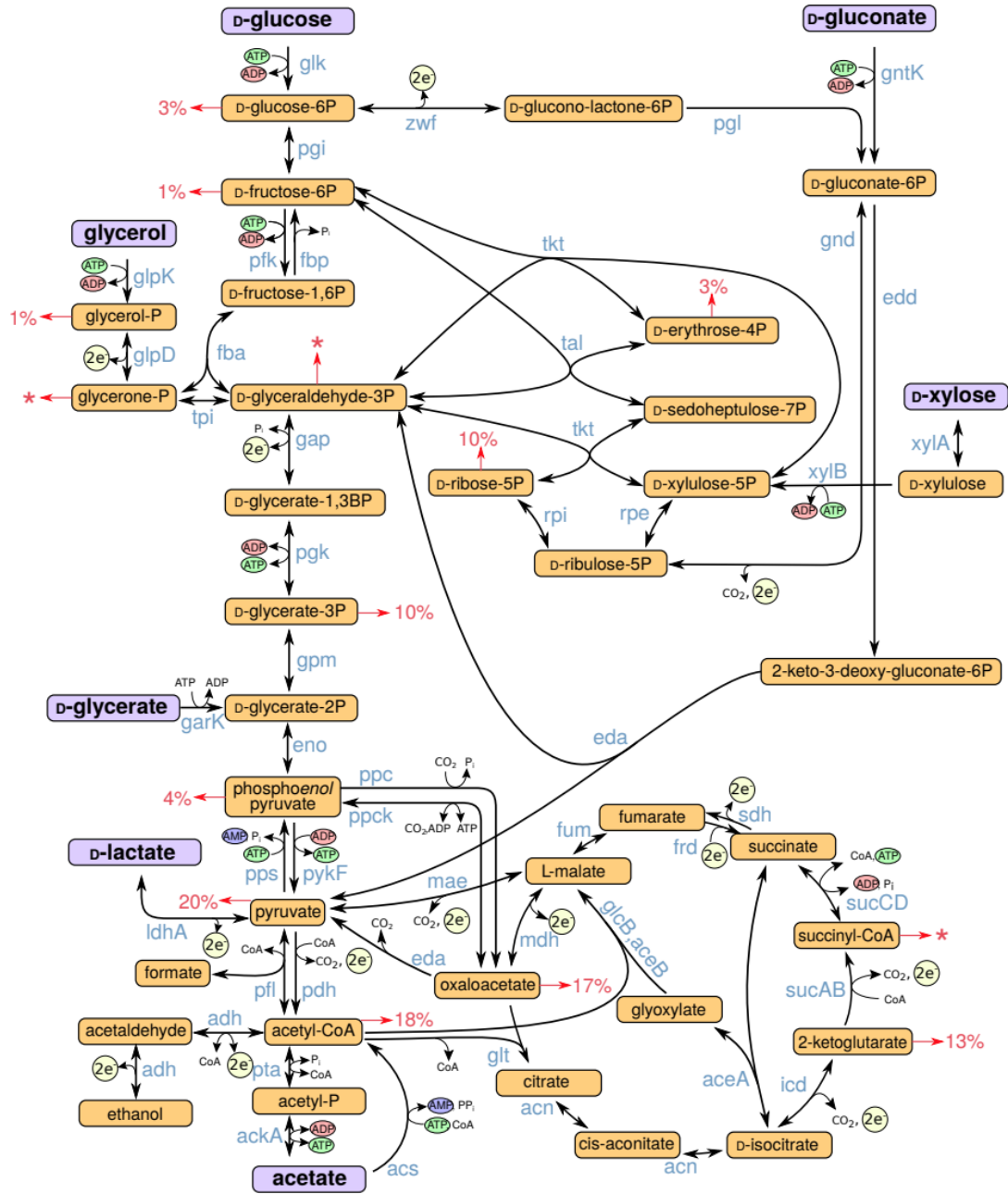


Figure 1.2: A map of central carbon metabolism in *E. coli* bacteria – The diagram shows the reactions, metabolites, co-factors, and enzymes, as well as a few selected carbon sources and their catabolic pathways (The Economic Cell Collective 2025).

$S\mathbf{v}$ will give us the rate of change in the concentration of each of our metabolites. Since this is a concentration instead of molar amounts, we also have to include a dilution factor (which we derived in Equation (1.6)). From this, we can create the following equation:

$$\dot{\mathbf{y}} = S\mathbf{v} - \epsilon\mu\mathbf{y}. \quad (1.10)$$

An important thing to note is that the stoichiometry matrix, S , will contain all of the reactions occurring inside of the cell, but it will also include other interactions that may not be reactions in a chemical sense. In order to provide external metabolites to the system, we will have exchange reactions and exchange fluxes, much like in Equation (1.9). In a cell, we may have different components of our cell, such as mitochondria (generators of ATP) or cytosol (fluid within the cell), and it may take time to transport metabolites between these components. So, we will consider the same chemical compound in different components of the cell as different metabolites. We will allow these metabolites to switch between each other through transport reactions. The last reaction we need is the biomass reaction, which again will represent the average metabolite demand to construct the precursors that make up biomass.

Combining this together gives us our generic model:

$$\begin{aligned} \dot{\mathbf{x}} &= -S_{exch}\mathbf{v}X, \\ \dot{\mathbf{y}} &= S\mathbf{v} - \epsilon\mu\mathbf{y}, \\ \dot{X} &= \epsilon\mu X, \end{aligned} \quad (1.11)$$

where S_{exch} is the component of the stoichiometric matrix S corresponding to exchange reactions, X refers to biomass and $\mathbf{x}$ refers to extracellular metabolites.

An important assumption that we have made is that our cell is ‘well-mixed’, in the sense that the concentrations of each of our metabolites (per component of the cell) is constant throughout the whole cell. This means that we will not have to worry about spatial differences in our system (but an even more complicated model may include this as a factor).

We want to understand how reactions occur and why specific reactions occur within a cell, since this may allow us to reduce the complexity of our problem. Many reactions are catalysed by enzymes. This means that the rate of these reactions will usually be very small, except when an enzyme is present - which allows the reaction to occur at a much faster rate. An enzyme is a long chain of amino acids, which must be wrapped in a specific structure to work correctly. This means that enzymes take up a large amount of space, and each enzyme is designed for a small number of reactions. So, the amount of each enzyme we can have within our cell is constrained, which in turn constrains the rates of each of our reactions. From physical experimentation we can calculate average enzyme concentrations, and use this to bound some of the variables in our model.

Even still, this system is very difficult to solve due to the many interacting dynamics and the fluxes $\mathbf{v}$ being dependent on many (sometimes unknown) factors.

In the next two sections of this chapter, we will demonstrate common ways of approaching these systems.

1.2 Flux Balance Analysis

In this section we will introduce the method of flux balance analysis (FBA). The underpinning assumption behind this method is to consider a steady state solution and try to calculate the fluxes between the reactions as the only variables. This will allow us to see how an organisms metabolism might occur in a steady state situation, contrived within a laboratory or industrial setting such as in a bioreactor with continuous input and output.

First, we will assume that the effect of dilution is negligible on our small molecules. When a cell increases in size by producing biomass, the concentrations of each of the metabolites would decrease since we have the same quantity in a larger volume. This would affect our system, but we will assume that compared to the rates of consumption and production, it is a negligible effect. Mathematically, this means $-\epsilon\mu\mathbf{y} = \mathbf{0}$.

So, under our steady state assumption of $\dot{\mathbf{y}} = \mathbf{0}$, we will have $S\mathbf{v} = \mathbf{0}$, which is an (under-determined) system of equations to be solved. We will also make the assumption that external metabolites are under a steady state condition. This may be for several reasons, such as dilution, constant supply/removal, or simply concentrations being so large the rate of change is negligible. So, solving this system of equations will give us a balanced set of fluxes through each of our reactions. Unfortunately, because this system is often underdetermined, we will have (infinitely) many potential solutions. To reduce the number of these solutions, we will start by applying constraints to the system.

Most of these additional constraints are motivated on physical grounds. The rate at which we can provide external metabolites to our cell will be bounded, since it requires a physical process, such as diffusion. The rate at which we can transport metabolites between components will also be bounded, since this may require transport molecules which will be limited by the size of the cell. We may also have an upper bound on the uptake rates of our extracellular substrates, due to Michaelis-Menten kinetics (The Economic Cell Collective 2025), which we will discuss more in Section 1.3.

We will now return to our system, applying these constraints to decrease the size of the set of solutions. Currently, our fluxes imply that all of our reactions can occur in either direction. Instead, we can split our reactions into forward fluxes and reverse fluxes. We can decompose each flux v_i into a forward v_i^+ and reverse v_i^- with $v_i^+, v_i^- \geq 0$ as $v_i = v_i^+ - v_i^-$, which leads to

$$0 = S\mathbf{v} = S\mathbf{v}^+ - S\mathbf{v}^- = (S \quad -S) \begin{pmatrix} \mathbf{v}^+ \\ \mathbf{v}^- \end{pmatrix}, \quad (1.12)$$

where $\mathbf{v}^+ = (v_1^+, \dots, v_n^+)^T$, and $\mathbf{v}^- = (v_1^-, \dots, v_n^-)^T$. Since we also know that $\mathbf{v}^+, \mathbf{v}^- \geq 0$, we can write our stoichiometry (Equation (1.10)) and non-negativity constraints together as

$$\begin{pmatrix} S & -S \\ -S & S \\ I & 0 \\ 0 & I \end{pmatrix} \begin{pmatrix} \mathbf{v}^+ \\ \mathbf{v}^- \end{pmatrix} \geq \mathbf{0} \quad (1.13)$$

where I is the identity matrix. We also want to add constraints to our fluxes to account for the assumptions we made earlier. By a slight abuse of notation, we re-define $\mathbf{v} = (\mathbf{v}^+ | \mathbf{v}^-)^T$ and add these constraints in the following way

$$A\mathbf{v} \geq \mathbf{b}, \quad (1.14)$$

with

$$A = \begin{pmatrix} S \\ -S \\ I \\ G \end{pmatrix}, \quad \mathbf{b} = \begin{pmatrix} \mathbf{0} \\ \mathbf{0} \\ \mathbf{0} \\ \mathbf{h} \end{pmatrix}. \quad (1.15)$$

The matrix G and the vector $\mathbf{h}$ will encapsulate our physical constraints on the bounds of fluxes for individual reactions. For example, if we have an upper bound on a specific flux $v_i \leq v_i^{ub}$, we can write this as a row of $G\mathbf{v} \geq \mathbf{h}$ in the form

$$(0 \quad \dots \quad 0 \quad -1 \quad 0 \quad \dots \quad 0) \mathbf{v} \geq (0 \quad \dots \quad 0 \quad -v_i^{ub} \quad 0 \quad \dots \quad 0)^T. \quad (1.16)$$

Since we have decomposed our fluxes into forwards and reverse directions, all constraints on our system are now in the form of upper bounds, since a negative lower bound would be converted to a positive upper bound on the reverse flux.

This model encapsulates all possible fluxes through each of our reactions that our organism might partake in, accounting for biological constraints such as limited space for enzymes or transport molecules. Despite this, this system will still often have many solutions so how do we know which one(s) are preferable for an organism?

To answer this question, we will introduce an ‘objective function’. This objective function will be a linear combination of fluxes and we will define an optimal solution as one that maximises this objective function over our set of feasible solutions (we could also minimise but this is equivalent, see Appendix A). So, our final problem is written as

$$\max_{\mathbf{v} \in \mathbb{R}^{2n}} \mathbf{c}^T \mathbf{v}, \quad (1.17)$$

subject to

$$A\mathbf{v} \geq \mathbf{b}, \quad (1.18)$$

where $\mathbf{c}$ is a vector containing the coefficients in our linear objective function. This is a linear programming (LP) problem and the method of constructing a model in this way is known as flux balance analysis (FBA). We have one last issue to solve with this model, that being the choice of our objective function. The objective function embodies the fundamental goal of an organism and allows us to choose the set of fluxes which is the best for this specific goal. Considering how organisms develop through survival of the fittest, a reasonable assumption to make is that an organisms primary goal will be to survive and reproduce. In order to do this, it will need to maximise the rate of biomass production (growth rate), e.g. μ in our small example from the previous section. So, we will use this as our objective function most of the time.

This is one way of considering the kinetic model of microbial metabolism, but it is heavily restrained by the steady state assumption. Next, we will show how we can dynamically extend this method, to account for a quasi-steady state system.

1.3 Dynamic Flux Balance Analysis

The FBA model is very good at predicting how a microorganism might behave in a steady-state system, when the environment is unchanging. However, in nature and often in industrial or laboratory conditions the environment is constantly changing. So, these organisms will need to be able to adapt and respond to different stresses and changes in extracellular metabolite concentrations (e.g. decrease in the concentration of a growth substrate as the organism consumes it). In industry, microorganisms are commonly grown in ‘continuous cultures’ or ‘batch cultures’. In continuous cultures, there is a constant in-flow of energy sources such as glucose and a constant out-flow of waste products such as ethanol. This system is perfect for being modelled using FBA, since it will quickly reach a steady-state and the rate of change of metabolite concentrations will become zero. Batch cultures on the other hand are self-contained systems ran over a set period of time. Because of this, the energy sources are limited and metabolic products will accumulate over time so we will not reach a steady-state solution until all growth substrates are depleted, when growth stops since the organism can no longer reproduce. Other industrial scenarios where the environment can change in time include temporal control of the parameters in a bioreactor (e.g. changes in temperature, feed rate, pH). Therefore, it will be very useful to extend our FBA model to account for a changing environment, which leads us to dynamic flux balance analysis (dFBA).

Recall our generic model for the dynamics of a microbial system:

$$\begin{aligned}\dot{\mathbf{x}} &= -S_{exch}\mathbf{v}X, \\ \dot{\mathbf{y}} &= S\mathbf{v}, \\ \dot{X} &= \epsilon\mu X,\end{aligned}\tag{1.19}$$

where again we have assumed that the effect of dilution is negligible.

Seeing as the parameter ϵ will be relatively small, we can consider the time separation limit as $\epsilon \rightarrow 0$. Since $\alpha\epsilon$ will remain constant, we will also have $\alpha \rightarrow \infty$. For simplicity, we can take $\alpha\epsilon = 1$. If we are in this time scale, we can consider a quasi-steady state solution. This is the result of using Tikhonov’s theorem, but we will not go into further details here (see Waldherr, Oyarzún, and Bockmayr (2014)). In essence, this is because the intracellular dynamics occur relatively quickly and will reach a steady state much faster than the extracellular dynamics. With this condition, we can assume the intracellular dynamics will be equal to zero ($\dot{\mathbf{y}} = 0$) and the extracellular dynamics will be as before. With this we can instead write our system of ODEs as a DAE (Differential-Algebraic system of Equations):

$$\begin{aligned}\dot{\mathbf{x}} &= -S_{exch}\mathbf{v}X, \\ S\mathbf{v} &= 0, \\ \dot{X} &= \epsilon\mu X.\end{aligned}\tag{1.20}$$

In order to obtain the fluxes, $\mathbf{v}$ (which includes μ), in this system, we can think of it as an embedded FBA problem. To do this, we will again require an objective function in order to select a solution from our underdetermined system $S\mathbf{v} = 0$, as well as constraints on the fluxes.

Since the bounds that we applied in Section 1.2 are dependent on the extracellular metabolites, we will have a ‘dynamic LP problem’, since it is now time dependent. Typically, this is done using dynamic bounds on the exchange reactions. A common choice is to apply upper bounds on the uptake rates. These bounds are of the form

$$-\frac{V_{i,max}x_i}{K_M + x_i} \leq v_i \leq 0,\tag{1.21}$$

where $V_{i,max}$ and K_M are the common parameters of the Michaelis-Menten kinetics (this is technically an upper bound since we have decomposed our fluxes into forward and reverse fluxes as before).

Putting these components together provides the ‘standard’ dFBA model, presented in Figure 1.3. This model allows us to see how a microorganism might influence and be influenced by its environment under the assumption that its goal is to maximise its growth rate subject to the extracellular environment at any moment in time, although alternative objective functions have also been considered. Since its conception (Mahadevan, Edwards, and Doyle 2002), dFBA has had a lot of success enabling researchers to build and simulate dynamic models of metabolism. However, as we will see in Chapter 2, care must be taken in simulating dFBA models, since numerical instabilities can often arise. Therefore, there is a need for more advanced algorithms to deal with these issues.

The standard dFBA model is incomplete since it assumes that the microorganism’s goal will remain constant at different times, and importantly under different conditions. Recently, the standard dFBA model has been extended to include the possibility that the growth strategy of the organism may itself vary in time as environmental conditions change (Henriques et al. 2021) (Moimenta et al. 2025). This is particularly relevant in an industrial setting, where there may be a trade-off between the maximal growth rate and the maximum rate of bio-production (Meadows et al. 2010). By introducing a dynamic objective function, we can model

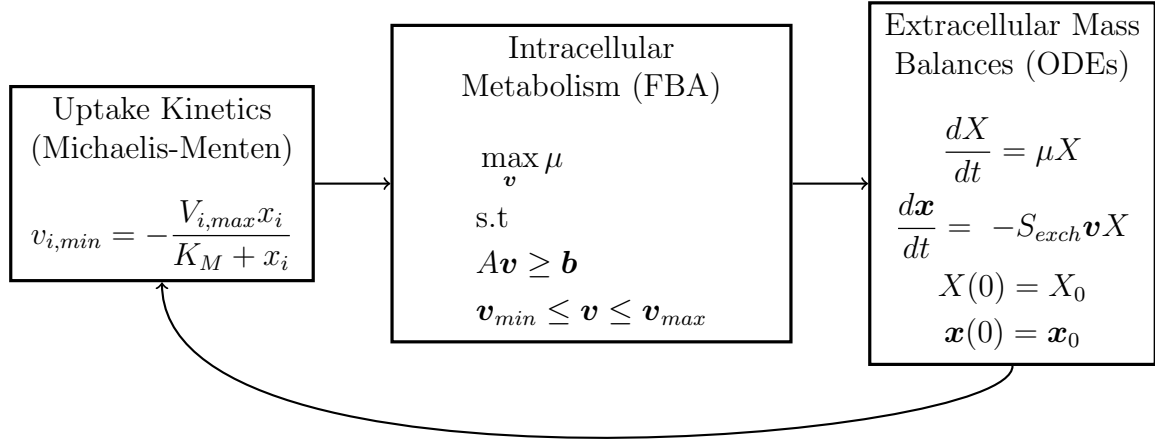


Figure 1.3: dFBA Model, represented by its three interacting components. In this we have the FBA model, the system of ODEs and the dynamic bounds on the LP variables.

how a microorganism might adapt its growth strategy in response to a dynamically changing environment.

To model this situation, we will consider two objectives μ_1, μ_2 (as parts of our vector of fluxes, $\mathbf{v}$) that we will switch between depending on the kinetic variables in the system. To do this, we will introduce a bounded variable $0 \leq \phi \leq 1$ that is a function of the extracellular metabolite concentrations. Our dynamic objective function is then given by a weighted trade-off of the two objectives in the form

$$z(t, \mathbf{x}(t)) = \max_{\mathbf{v}} (\mu_1 \phi + \mu_2 (1 - \phi)). \quad (1.22)$$

The general form of this model, with a dynamic objective function constructed in this way, is a relatively recent extension. It was introduced by Moimenta et al. (2025) who used a multiphase objective function to model how yeast might adapt to changing nitrogen concentrations in the environment. In their case, they choose μ_1 to represent the rate of protein production while μ_2 is the rate of ATP production, with ϕ an increasing function of extracellular nitrogen sources. As described by the authors, this choice of dynamic objective function provided the best fit of the model to their experimental data. However, efficient simulation algorithms to deal with these extended DFBA models have not yet been developed.

1.4 Report Summary

The layout of the remainder of this report is as follows.

Chapter 2 will focus on introducing the foundational mathematical background for FBA and dFBA problems. We will also introduce the efficient dFBA simulation algorithm introduced by Harwood, Höffner, and Barton (2016) (which we will refer to as the Harwood et al. algorithm for brevity). To demonstrate the effectiveness of this algorithm we will also present the ‘direct’ method which is simpler to implement but has issues that may make it unsuitable for solving these problems. We will also outline some of the relevant LP theory required to understand the algorithm. Understanding this theory in detail will prove useful since it will then allow us to motivate our extension of the Harwood et al. algorithm to efficiently and robustly simulate dFBA problems with variable objective functions, which will form the main, novel result of this thesis.

Chapter 3 will focus entirely on this extended algorithm, which is capable of solving dFBA problems with multiphase objective functions. We will explain how it works, justify that it is correct and demonstrate its application on a few toy examples. To our knowledge, this extended algorithm represents entirely novel contribution to the field.

In Chapter 4 we will then present a biological example of a real-world dFBA problem with a variable objective function. This will include both the ‘standard’ objective of maximising biomass and a multiphase objective that accounts for an organism attempting to protect itself from toxicity by minimising ethanol production.

We conclude the report with some general remarks and opportunities for future extensions.

Chapter 2

Mathematical Theory of Standard DFBA Problems

2.1 Required LP Theory

In Equation (1.17) and Equation (1.18), we presented the FBA model as a linear programming (LP) problem. We will now formally state the LP problem in mathematical terms, then explain how we can relate it back to the underpinning biology.

Let $A \in \mathbb{R}^{m \times n}$, $\mathbf{c} \in \mathbb{R}^n$, $\mathbf{b} \in \mathbb{R}^m$. The goal of our LP problem is to find $\mathbf{v}^* \in \mathbb{R}^n$ such that it satisfies the following conditions:

$$\begin{aligned} \mathbf{c}^T \mathbf{v}^* &= \max_{\mathbf{v} \in \mathbb{R}^n} \mathbf{c}^T \mathbf{v}, \\ \text{subject to } A\mathbf{v} &\geq \mathbf{b}, \\ \mathbf{v} &\geq \mathbf{0}. \end{aligned} \tag{2.1}$$

In Appendix A we will show how any LP problem can be written in this way.

To convert between this LP problem and our FBA model, we can write $\mathbf{v} = (\mathbf{v}^+ | \mathbf{v}^-)$ where $\mathbf{v}^+$ is the vector of fluxes through the forward direction reactions and $\mathbf{v}^-$ is the vector of fluxes through the reverse direction reactions. We also have

$$A = \begin{pmatrix} S' \\ -S' \\ G \end{pmatrix}, \mathbf{b} = \begin{pmatrix} \mathbf{0} \\ \mathbf{0} \\ \mathbf{h} \end{pmatrix}, \tag{2.2}$$

where $S' = (S | -S)$ is the stoichiometric matrix, containing the coefficients of each metabolite in each reaction, and G and $\mathbf{h}$ together encode the upper and lower bounds of our system - derived from the constraints on enzyme levels, for example.

When we introduced our FBA model in Section 1.2, we said that there were m metabolites and n reactions. With this representation of our model, we instead have m rows and n columns in our matrix A (since this is standard notation). Since our vector of fluxes includes forward and reverse directions, we have $n/2$ reversible reactions in our original system. Also, letting r be the number of rows in S' and g be the number of rows in G , we have $m = 2r + g$. So, in our original system (and still in this system) we have r metabolites.

Since our implementation uses GLPK (Free Software Foundation 2013), a package to solve the LP problems, we will want to rewrite our system in Equation (2.1) in a way that is interpretable by the package. First, we will move away from the traditional way of encoding the upper and lower bounds into G and $\mathbf{h}$. Instead, we will write $\mathbf{l} \leq \mathbf{v} \leq \mathbf{u}$, where the elements of $\mathbf{l}$ and $\mathbf{u}$ are allowed to be arbitrarily large to represent unconstrained variables. We may still have additional, more complicated constraints embedded in G and $\mathbf{h}$ but we have removed the

simplest ones. Next, we will introduce auxiliary variables by rewriting our linear constraints $A\mathbf{v} \geq \mathbf{b}^1$ as $A\mathbf{v}_S = \mathbf{v}_R$; where we call $\mathbf{v}_R$ the vector of auxiliary variables and $\mathbf{v}_S$ the vector of structural variables. We can also write these constraints as $(I| - A)\mathbf{v} = \mathbf{0}$. With this choice, we will then have $\mathbf{v}_R \geq \mathbf{b}$, which we can include into our vector of lower bounds $\mathbf{l}$.

We will say $\mathbf{v}_S \in \mathbb{R}^{n_S}$, $\mathbf{v}_R \in \mathbb{R}^{n_R}$ and, by a slight abuse of notation, we will write $\mathbf{v} = (\mathbf{v}_R|\mathbf{v}_S)$, $n = n_S + n_R$. m will be the number of rows remaining in A (after we removed some). So, we will have $\mathbf{v} \in \mathbb{R}^n$, $A \in \mathbb{R}^{m \times n_S}$, $\mathbf{c} \in \mathbb{R}^{n_S}$ and $\mathbf{l}, \mathbf{u} \in \mathbb{R}^n$. We will also write $\mathbf{l} = (\mathbf{l}_R|\mathbf{l}_S)$, $\mathbf{u} = (\mathbf{u}_R|\mathbf{u}_S)$ in order to separate the auxiliary and structural bounds. Now, we have finally formulated the LP problem which we will be solving. We want to find $\mathbf{v}^* \in \mathbb{R}^n$ such that it satisfies the following conditions:

$$\begin{aligned} (\mathbf{0}|\mathbf{c})^T \mathbf{v}^* &= \max_{\mathbf{v} \in \mathbb{R}^n} (\mathbf{0}|\mathbf{c})^T \mathbf{v}, \\ \text{subject to } (I| - A)\mathbf{v} &= \mathbf{0}, \\ \mathbf{l} &\leq \mathbf{v} \leq \mathbf{u}. \end{aligned} \tag{2.3}$$

To solve this problem, we will first assume that A has full rank, which means that $(I| - A)$ will have full rank as well (we explain this in further detail in Appendix A). This is a necessary assumption so that we can transform this problem using Karush-Kuhn-Tucker (KKT) conditions. These conditions are the following:

$$\nabla_{\mathbf{v}} \mathcal{L} := (\mathbf{0}|\mathbf{c})^T - (I| - A)^T \boldsymbol{\pi} - \boldsymbol{\lambda}_l - \boldsymbol{\lambda}_u = \mathbf{0}, \tag{2.4}$$

$$(I| - A)\mathbf{v} = \mathbf{0}, \tag{2.5}$$

$$\mathbf{l} \leq \mathbf{v} \leq \mathbf{u}, \tag{2.6}$$

$$\boldsymbol{\lambda}_l \geq \mathbf{0}, \quad \boldsymbol{\lambda}_u \leq \mathbf{0}, \tag{2.7}$$

$$(\lambda_l)_k (v_k - l_k) = 0, \quad (\lambda_u)_k (v_k - u_k) = 0, \quad \text{for } k = 0, 1, \dots, n, \tag{2.8}$$

where:

- $\boldsymbol{\pi}$ is an n -vector of Lagrange multipliers corresponding to the linear constraints represented by $(I| - A)\mathbf{x} = \mathbf{0}$.
- $\boldsymbol{\lambda}_l$ is an n -vector of Lagrange multipliers corresponding to the lower bounds, $\mathbf{l} \leq \mathbf{v}$.
- $\boldsymbol{\lambda}_u$ is an n -vector of Lagrange multipliers corresponding to the upper bounds, $\mathbf{v} \leq \mathbf{u}$.

Since our problem is linear, we have that $\nabla_{\mathbf{v}}^2 \mathcal{L} = \mathbf{0}^2$. This means that a solution to our KKT conditions is an optimal solution to our LP problem (this is a known result from optimisation theory). Essentially, if we can find $\mathbf{v}^*, \boldsymbol{\pi}^*, \boldsymbol{\lambda}_l^*, \boldsymbol{\lambda}_u^* \in \mathbb{R}^n$ that satisfy our KKT conditions, then $\mathbf{v}^*$ is necessarily an optimal solution to our LP problem given in Equation (2.1).

We want to transform these conditions so that they are in a form that is easier to handle. To do this, we will introduce the concept of a basic solution.

Definition 1 Consider an LP problem with m linear constraints and n variables. A basic solution to this problem $\mathbf{v}$ is in the form $\Pi\mathbf{v} = \Pi(\mathbf{v}_R|\mathbf{v}_S) = (\mathbf{v}_B|\mathbf{v}_N)$, where Π is a permutation matrix, $\mathbf{v}_B$ is an m -vector of basic variables and $\mathbf{v}_N$ is an $(n - m)$ -vector of non-basic variables. Non-basic variables are defined to be set at their upper or lower bounds. Since A has full rank, this means that basic variables are then fixed. We can also consider a basic solution to be an intersection of m hyper-planes, defined by our linear constraints and bounds.

¹ A and $\mathbf{b}$ are now smaller due to the removed constraints.

²This is the matrix of all zeros, instead of a vector of all zeros.

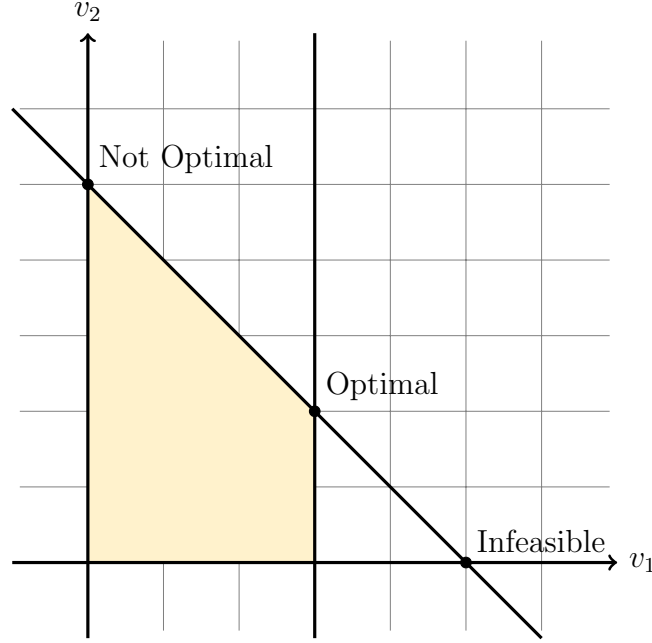


Figure 2.1: Basic solutions explained: This diagram represents an LP problem, where the yellow region (and its border) is the feasible set of solutions. Since this is in two dimensions, basic solutions are at the intersections of two constraints (lines). Three basic solutions have been labelled. The solution labelled ‘Infeasible’ is not in the feasible region so it is not feasible. The other two solutions are both feasible, but only one is optimal (for some fixed but not provided objective function).

This definition may not seem helpful at first, but it will allow us to reduce the problem significantly. We will also require the following result, since this will justify that looking at basic solutions is a worthwhile endeavour.

Theorem 2 *Suppose there exists an optimal solution to the LP problem as described in Equation (2.1). Then, there exists a basic solution to the LP problem which is optimal.*

In the case where the LP problem has a unique solution, this suggests that we can write this unique solution in the form of a basic solution (which is true). The point being, either the LP problem is unsolvable (which may be the case) or there exists an optimal basic solution. So, our goal now is to reduce the problem to find when a basic solution is optimal.

Consider a basic solution to our KKT conditions. We can write $\mathbf{v} = \Pi^T \Pi \mathbf{v}$ for some permutation matrix Π . From this, Equation (2.5) gives us

$$(I| - A)\Pi^T \Pi \mathbf{v} = (B|N) \begin{pmatrix} \mathbf{v}_B \\ \mathbf{v}_N \end{pmatrix} = \mathbf{0}, \quad (2.9)$$

where

$$(B|N) = (I|A)\Pi^T.$$

B is a square matrix which is necessarily non-singular, which means that B^{-1} must exist. Then, Equation (2.9) gives us

$$B\mathbf{v}_B + N\mathbf{v}_N = \mathbf{0}, \quad (2.10)$$

$$\implies \mathbf{v}_B = -B^{-1}N\mathbf{v}_N. \quad (2.11)$$

We define the simplex tableau as the matrix $\Xi := -B^{-1}N$, and we find that the basic variables $\mathbf{v}_B$ are uniquely determined by the non-basic variables $\mathbf{v}_N$ (which are fixed at their bounds).

We note that for a basic variable, Equation (2.11) is equivalent to Equation (2.5). This means that defining our basic variables in this way guarantees Equation (2.5) is satisfied. For Equation (2.6), we say that a basic solution for which each $\mathbf{l} \leq \mathbf{v} \leq \mathbf{u}$ is defined as primal feasible and move on since we cannot reduce this further. Any basic solution which satisfies both Equation (2.5) and Equation (2.6), will be in the feasible region of our LP problem - so it would be a potential choice of fluxes in the FBA problem. The next set of conditions are what requires such a feasible solution to be optimal.

Next, from Equation (2.8), we realise that either $\lambda_{l,k} = 0$ or $\lambda_{u,k} = 0$ for each $k = 0, 1, \dots, n$ ($\lambda_{l,k}$ is the k th element of the vector $\boldsymbol{\lambda}_l$). This is because we cannot have $v_k - l_k = u_k - v_k = 0$ for any variable³. Therefore, we can define $\boldsymbol{\lambda} = \boldsymbol{\lambda}_l + \boldsymbol{\lambda}_u$. Then, we multiply Equation (2.4) by Π and substitute in $\boldsymbol{\lambda}$ to get

$$\Pi(\mathbf{0}|\mathbf{c})^T - \Pi(I| - A)^T \boldsymbol{\pi} - \Pi \boldsymbol{\lambda} = \mathbf{0}. \quad (2.12)$$

We will separate the $\mathbf{c}$ and $\boldsymbol{\lambda}$ into basic and non-basic components by writing $\Pi(\mathbf{0}|\mathbf{c})^T = (\mathbf{c}_B|\mathbf{c}_N)^T$ and $\Pi \boldsymbol{\lambda} = (\boldsymbol{\lambda}_B|\boldsymbol{\lambda}_N)^T$. Then, using $\Pi(I| - A)^T = ((I| - A)\Pi^T)^T = (B|N)^T$, we get

$$(\mathbf{c}_B|\mathbf{c}_N)^T - (B|N)^T \boldsymbol{\pi} - (\boldsymbol{\lambda}_B|\boldsymbol{\lambda}_N)^T = \mathbf{0}, \quad (2.13)$$

$$\implies \begin{pmatrix} \mathbf{c}_B \\ \mathbf{c}_N \end{pmatrix} - \begin{pmatrix} B^T \\ N^T \end{pmatrix} \boldsymbol{\pi} - \begin{pmatrix} \boldsymbol{\lambda}_B \\ \boldsymbol{\lambda}_N \end{pmatrix} = \mathbf{0}. \quad (2.14)$$

This then provides us with the following system of simultaneous equations:

$$\begin{cases} B^T \boldsymbol{\pi} + \boldsymbol{\lambda}_B = \mathbf{c}_B, \\ N^T \boldsymbol{\pi} + \boldsymbol{\lambda}_N = \mathbf{c}_N. \end{cases} \quad (2.15)$$

Denoting $B^{-T} = (B^T)^{-1}$, we can solve the first equation for $\boldsymbol{\pi}$ to get

$$\boldsymbol{\pi} = B^{-T} \mathbf{c}_B - B^{-T} \boldsymbol{\lambda}_B, \quad (2.16)$$

and substituting this into the second equation, we get

$$\boldsymbol{\lambda}_N = \mathbf{c}_N - N^T \boldsymbol{\pi} = N^T B^{-T} \boldsymbol{\lambda}_B + (\mathbf{c}_N - N^T B^{-T} \mathbf{c}_B). \quad (2.17)$$

Using $N^T B^{-T} = (B^{-1}N)^T = -\Xi^T$, we can rewrite this as

$$\boldsymbol{\lambda}_N = -\Xi^T \boldsymbol{\lambda}_B + \mathbf{d}, \quad (2.18)$$

where

$$\mathbf{d} = \mathbf{c}_N + \Xi^T \mathbf{c}_B. \quad (2.19)$$

We call $\mathbf{d}$ the vector of reduced costs. Now, since we are considering a basic solution, and we require $l_k \neq v_k \neq u_k$ for each k where $v_k \in \mathbf{v}_B$, Equation (2.8) necessitates that $\boldsymbol{\lambda}_B = \mathbf{0}$.

Therefore, $\boldsymbol{\lambda}_N = \mathbf{d}$. For any basic solution, this equality is equivalent to Equation (2.4). A basic solution for which $d_k \geq 0$ for $(\mathbf{v}_N)_k$ set at its upper bound or $d_k \leq 0$ for $(\mathbf{v}_N)_k$ set at its lower bound is defined as dual feasible.

By choosing specifically $\boldsymbol{\lambda}_B = \mathbf{0}$, $\boldsymbol{\pi} = B^{-T} \mathbf{c}_B$ and $\boldsymbol{\lambda}_N = \mathbf{d}$ we will necessarily satisfy Equation (2.4) and Equation (2.8). If we also have that our solution is dual feasible (i.e. Equation (2.7) is satisfied), then our basic solution will be optimal as long as it is feasible. The value of the objective function for such a solution will be at least as large as the maximum of the objective function over the feasible set.

By reducing our scope of the problem from all possible solutions to just basic solutions, we have reduced the problem since three of the five KKT conditions will always be satisfied. We can summarise this information into the following theorem:

³If we did, either our problem was about to become infeasible or v_k is constant and we can remove it from the problem.

Theorem 3 *A basic solution of our LP problem is defined by the following properties:*

- $v_k = u_k$ or $v_k = l_k$ for each $v_k \in \mathbf{v}_N$,
- $\mathbf{v}_B = \Xi \mathbf{v}_N$,
- $\boldsymbol{\pi} = B^{-T} \mathbf{c}_B$,
- $\boldsymbol{\lambda}_B = 0$,
- $\boldsymbol{\lambda}_N = \mathbf{d}$, where $\mathbf{d} = \mathbf{c}_N + \Xi^T \mathbf{c}_B$.

This is an optimal solution of our LP problem (and a solution to our KKT conditions) if it is both primal feasible and dual feasible:

- $\mathbf{l}_B \leq \mathbf{v}_B \leq \mathbf{u}_B$,
- $\lambda_{N,k} \geq 0$ for each $v_k \in \mathbf{v}_N$ set at its upper bound and $\lambda_{N,k} \leq 0$ for each $v_k \in \mathbf{v}_N$ set at its lower bound.

We will require this LP theory to justify the Harwood et al. algorithm in Section 2.4. For now, we will demonstrate a small toy example of a LP problem. We will slowly build up this example, making it more complicated as we introduce more ideas but to start with we will have the following LP problem (this is the same as the example in the GLPK notation (Free Software Foundation 2013)):

$$\begin{aligned}
 & \max_{\mathbf{v} \in \mathbb{R}^3} 10v_1 + 6v_2 + 4v_3, \\
 & \text{subject to } v_1 + v_2 + v_3 \leq 100, \\
 & \quad 10v_1 + 4v_2 + 5v_3 \leq 600, \\
 & \quad 2v_2 + 2v_2 + 6v_3 \leq 300, \\
 & \quad v_1, v_2, v_3 \geq 0.
 \end{aligned} \tag{2.20}$$

By introducing our structural variables as before, this transforms into the following problem:

Example 1

$$\begin{aligned}
 & \max_{\mathbf{v} \in \mathbb{R}^3} 10v_1 + 6v_2 + 4v_3, \\
 & \text{subject to } v_1 + v_2 + v_3 = v_4, \\
 & \quad 10v_1 + 4v_2 + 5v_3 = v_5, \\
 & \quad 2v_2 + 2v_2 + 6v_3 = v_6, \\
 & \quad v_1, v_2, v_3, v_4, v_5, v_6 \geq 0, \\
 & \quad v_4 \leq 100, v_5 \leq 600, v_6 \leq 300.
 \end{aligned} \tag{2.21}$$

We will frequently refer to the simplex method, which is a method of pivoting from basis to basis until we reach an optimal basis. It also allows us to calculate the simplex tableau for a basis, but further detail is not required here. We will direct the interested reader to Murty (1983).

By applying the aforementioned simplex method, we find that the optimal solution to this problem is $\mathbf{v}^* = (\frac{100}{3}, \frac{200}{3}, 0, 100, 600, 200)^T$. This solution has basic variables v_1, v_2 and v_6 . v_3 is a non-basic variable set at its lower bound and v_4 and v_5 are non-basic variables set at their upper bounds. We will also have the following simplex tableau (by calculating $-B^{-1}N$):

$$\Xi_1 = \begin{pmatrix} -\frac{1}{6} & -\frac{2}{3} & \frac{1}{6} \\ -\frac{5}{6} & \frac{5}{3} & -\frac{1}{6} \\ 4 & 2 & 0 \end{pmatrix} \quad (2.22)$$

We can check that this is indeed an optimal solution by verifying that it is both primal and dual feasible (by Theorem 3). Clearly, it is primal feasible since all variables are non-negative (in our other notation we have $\mathbf{l} = \mathbf{0}$ and $\mathbf{u}$ is arbitrarily large). To check dual feasibility, we want to check the conditions on our reduced costs. We have $\mathbf{d} = \mathbf{c}_N + \Xi_1^T \mathbf{c}_B = (-\frac{8}{3}, \frac{10}{3}, \frac{2}{3})^T$. Since our first non-basic variable is at its lower bound, we require $d_1 \leq 0$ which we have. Our remaining two non-basic variables are set at their upper bounds, so we require $d_2, d_3 \geq 0$ which again we have. Therefore, we can be confident that this is an optimal solution to our LP problem by applying Theorem 3.

2.2 Formulation of the Standard dFBA Problem

Now we will return to dFBA, the dynamic extension of FBA. From Section 1.3, we know that we require three components to build a dFBA model. We will need an FBA problem, written as an LP problem, developed from the stoichiometry matrix. We will also need kinetic variables, representing extracellular concentrations such as glucose or oxygen levels. Finally, we will need dynamic bounds, such as those developed from the Michaelis-Menten equations. We will now give the formal problem statement for an initial value problem (IVP) with embedded LP problem, then relate this to the standard dFBA problem.

Let $D_t \subset \mathbb{R}$, $D_x \subset \mathbb{R}^{n_x}$ and $D_z \subset \mathbb{R}$ be non-empty open sets, representing the domains of time, kinetic variables and the objective function's output, respectively. n_x is the number of kinetic variables in our system. Let $\mathbf{f} : D_t \times D_x \times D_z \rightarrow \mathbb{R}^{n_x}$, $\mathbf{q} : D_t \times D_x \rightarrow \mathbb{R}^m$, and $A \in \mathbb{R}^{m \times n_v}$, be given. $\mathbf{f}$ is a function that calculates the value of our kinetic values at a specific time, $\mathbf{q}$ is a function that calculates the right hand side of our LP problem at a specific time and A is our stoichiometry matrix. First, let $\mathbb{R} = \mathbb{R} \cup \{-\infty\} \cup \{\infty\}$, then let $\hat{z} : \mathbb{R}^m \rightarrow \mathbb{R}$ be such that

$$\begin{aligned} \hat{z}(\mathbf{b}) &= \sup_{\mathbf{v} \in \mathbb{R}^{n_v}} \mathbf{c}^T \mathbf{v} \\ &\text{subject to } A\mathbf{v} = \mathbf{b}, \\ &\mathbf{v} \geq \mathbf{0}. \end{aligned} \quad (2.23)$$

Subsequently, define

$$\begin{aligned} F &\equiv \{\mathbf{b} \in \mathbb{R}^m : -\infty < \hat{z}(\mathbf{b}) < +\infty\}, \\ K &\equiv \mathbf{q}^{-1}(F). \end{aligned} \quad (2.24)$$

F represents the set of possible right hand side vectors, $\mathbf{b}$, such that the LP is feasible and bounded. $K \subset D_t \times D_x$ is the set of times and kinetic variable values for which the LP is feasible and bounded.

Then a dFBA problem is formulated as an initial value problem (IVP) in ODEs: given an initial time $t_0 \in D_t$ and initial conditions $\mathbf{x}_0 \in D_x$, we seek an interval $[t_0, t_f] = I \subset D_t$, and an absolutely continuous function $\mathbf{x} : I \rightarrow D_x$ which satisfy

$$\dot{\mathbf{x}}(t) = \mathbf{f}(t, \mathbf{x}(t), z(t, \mathbf{x}(t))), \quad \text{a.e. } t \in I, \quad \mathbf{x}(t_0) = \mathbf{x}_0, \quad (2.25)$$

where $z : K \rightarrow \mathbb{R} : (t, \mathbf{x}) \mapsto \hat{z}(\mathbf{q}(t, \mathbf{x}))$. Such an I and $\mathbf{x}$ are called a solution of Equation (2.25).

Harwood, Höffner, and Barton (2016) prove several results about this problem, including that it is well-defined (in the sense that a solution exists). We will neglect these results for now, as we will touch on them later instead when we discuss the extended algorithm. Importantly, they prove the problem has a unique solution when applied to a lexicographic LP problem. To make the problem easier we have reduced the problem to a traditional LP problem, but this means that the only value from the LP problem which the kinetic variables depend on is that of the objective function. So, the problem stated above does in fact have a unique solution, but once we try to extract further fluxes (such as from exchange reactions), we run into an issue of uniqueness. In a lexicographic method, each flux that is required in our system of ODEs is written in a series of consecutive objective functions - so that each of these fluxes is guaranteed to attain a unique value. With our simplification, it may be that multiple choices of fluxes lead to an optimal objective (such as maximising biomass) but the fluxes through the exchange reactions may not be the same between solutions. This may cause kinetic variables to have different values between solutions to the IVP, which suggests the solution of Equation (2.25) that we are looking for may not be unique. Since we are primarily interested in the extended dFBA problem with a variable objective, the issue of non-uniqueness (which also applies in that case) is beyond the scope of what we consider here, and so for now we will simply overlook it. In our toy models this will not be a problem, since the only flux we are extracting from the LP problem is the objective value. However, future work will focus on extending our new framework to incorporate lexicographic optimisation.

In this notation, our FBA problem remains mostly the same except for the change from finding the maximum of the objective function to finding the supremum. This difference allows us to define F neatly. The equation $-\infty < \hat{z}(\mathbf{b}) < +\infty$ allows us to implicitly embed the necessity that the problem is bounded and feasible. If the problem is unbounded (in the sense that we can find a sequence of solutions whose objective values tend to infinity), then the supremum will be equal to $+\infty$. If our problem is infeasible (in that there are no solutions to the constraints), then the supremum will be equal to $-\infty$. This difference is then removed by the definition of $z(\mathbf{q}(t, \mathbf{x}))$ since it is only evaluated on the domain K , where the supremum provides a real value. This means we can then return from the supremum to our usual preferred maximum. Then, we allow $\mathbf{b}$ to change, depending on the values of the kinetic variables and time. This is necessary since our uptake rates are dependent on the concentrations of our extracellular metabolites. Finally, the system of ODEs which represent how the concentrations of these metabolites change over time is given by $\dot{\mathbf{x}} = \mathbf{f}$, where $\mathbf{f}$ will contain the necessary mass balances.

In terms of structural variables, we can rewrite the problem into the following form:

$$\begin{aligned} \dot{\mathbf{x}}(t) &= \mathbf{f}(\mathbf{x}(t), z(\mathbf{x}(t))), \\ \mathbf{x}(t_0) &= \mathbf{x}_0, \\ \text{where } z(\mathbf{s}) &= \max_{\mathbf{v} \in \mathbb{R}^n} (\mathbf{0} | \mathbf{c})^T \mathbf{v}, \\ \text{subject to } (I | A) \mathbf{v} &= \mathbf{0}, \\ \mathbf{l}(\mathbf{s}) &\leq \mathbf{v} \leq \mathbf{u}(\mathbf{s}). \end{aligned} \tag{2.26}$$

We will now extend our toy model given in Equation (2.20) to be representative of a dFBA problem. This toy model is too simple to capture a biological system (which we will consider in Chapter 4) but will allow us to demonstrate our techniques. To do this we will add a kinetic variable $x \in \mathbb{R}$ that experiences exponential decay through the equation

$$\dot{x} = -\frac{z(x(t))}{z(x_0)} x(t)$$

and the initial conditions $t_0 = 0, x_0 = x(t_0) = 0$. Then, we will also introduce a bound on our equations which will become tighter over time. The bound on the second equation we will change from 600 to $600x$. Putting this together creates the following problem:

Example 2

$$\begin{aligned}
\dot{x}(t) &= f(x(t), z(x(t))) = -\frac{z(x(t))}{z(x_0)}x(t), \\
x(t_0) &= 1, \\
\text{where } z(s) &= \max_{v \in \mathbb{R}^3} 10v_1 + 6v_2 + 4v_3, \\
\text{subject to } v_1 + v_2 + v_3 &\leq 100, \\
10v_1 + 4v_2 + 5v_3 &\leq 600s, \\
2v_2 + 2v_2 + 6v_3 &\leq 300, \\
v_1, v_2, v_3 &\geq 0.
\end{aligned} \tag{2.27}$$

This is not a problem we can easily solve without developing further techniques. For the rest of this chapter we will demonstrate algorithms for solving these types of hybrid dynamic optimisation problems.

2.3 Direct Method for Solving Standard dFBA Problems

We will now present a commonly used technique to simulate the standard dFBA problem known as the ‘direct method’. This method involves breaking the time interval we want to simulate our problem over into discrete time steps. This allows us to find the optimal solution to our LP problem for every fixed time point, and use numerical integration techniques to update our dynamics for the next time step. This is summarised in Algorithm 1.

Algorithm 1 Direct Method

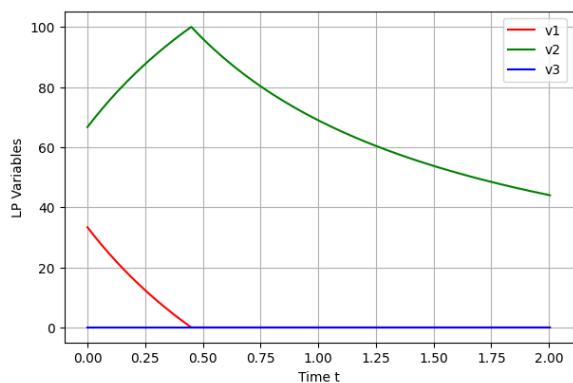
Input: Dynamic LP problem, initial values of kinetic variables $\mathbf{x}_0$, time step h , start time t_0 , finish time t_{max}

Output: Numerical approximation of dynamic LP problem over $[t_0, t_{max})$

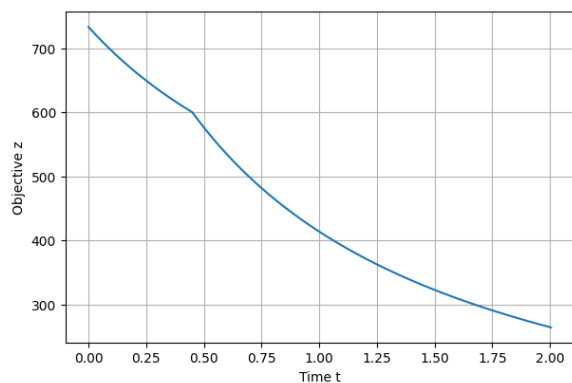
- 1: $t \leftarrow t_0$
- 2: $k \leftarrow 0$
- 3: **while** $t < t_{max}$ **do**
- 4: Solve the LP problem at time t for $\mathbf{v}_k, z_k$; if infeasible, break the loop
- 5: Integrate the ODE system over the time interval $[t, t + h]$ to calculate new values of kinetic variables $\mathbf{x}_{k+1}$
- 6: Update the bounds in the LP problem using the new values of the kinetic variables, $\mathbf{x}_k$
- 7: $t \leftarrow t + h$
- 8: $k \leftarrow k + 1$
- 9: **end while**
- 10: **return** all values of $\mathbf{v}_k, z_k, \mathbf{x}_k$

Before we demonstrate the numerical issues that can arise with the direct method, we will first apply it to our toy model introduced in the previous section (Equation (2.27)). We run the simulation of this problem over the time interval $[0, 2]$ and the results are seen in Figure 2.2.

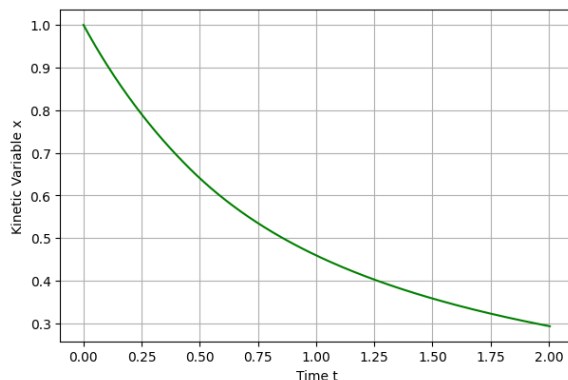
The benefit to using the toy model is that it is simple enough to fully describe the behaviour analytically, so that we can compare with simulations. We can think greedily in order to do this. This means that we want as much of the ‘best’ variable as possible, that is, the variable with the largest objective coefficient relative to its constraint coefficients. For the first and third constraints, this is v_1 and for the second constraint, this is v_2 . Thinking of the problem like this gives some insight into what is happening in our model. When the constraint for the second equation is large (i.e. initially), we can afford to choose v_1 since it is the best variable in terms of the other two constraints. However, v_2 is still a good second choice and as the constraint on the second equation tightens, v_2 being the best here means it becomes more viable. It is because of this that the value of v_2 increases, while the value of v_1 decreases. Once v_1 reaches 0, v_2 no longer has any room to continue increasing, so it must now start decreasing. We can see the moment that v_1 becomes 0 in Figure 2.2b since it is not a smooth curve at that point. Without more information, we cannot say much about the long term behaviour of the system, but we may hypothesise that v_2, z and x will continue to decrease - moving towards the steady state solution of $v = 0, z = x = 0$.



(a) LP Variables, $\mathbf{v}$



(b) Objective Value, z



(c) Kinetic Variable Value, x

Figure 2.2: Solving the toy model (Example 2) using the direct method presented in Algorithm 1.

Now we will look at why the direct method may run into numerical issues when solving dFBA problems. The first issue with this method is that the number of times we have to solve the LP problem is dependent on our time step, but so is the quality of our integration. This means we have a trade-off between the accuracy of our solution and the time it takes for our algorithm to run. Correctly choosing a value of h such that the problem can be solved quickly and accurately is a case of trial and error. In the case of Figure 2.2, a choice of $h = 0.01$ allowed for a numerically accurate solution but $h = 0.1$ did not. For larger models, the ‘ideal’ value of h will be very difficult to establish in general.

A more significant issue is that we may have numerical instability. Harwood, Höffner, and Barton (2016) presented an example problem where the direct method fails to find a solution to the problem regardless of the time step chosen. This example was shown using an explicit Euler step. Since our implementation of the direct method instead used RK4, a different Runge-Kutta technique, for numerical integration, we will show that this method also fails for the same reason. The problem is given as follows:

$$\begin{aligned}\dot{\mathbf{x}}(t) &= \mathbf{f}(\mathbf{x}(t), z(\mathbf{x}(t))) = \begin{bmatrix} 1 \\ x_2(t)z(\mathbf{x}(t)) - (x_2(t))^2 + 2x_1(t) \end{bmatrix}, \\ \mathbf{x}(0) &= \mathbf{0}, \\ \text{where } z(\mathbf{s}) &= \min_{v \in \mathbb{R}} v, \\ \text{subject to } s_1^2 &\leq v \leq s_2.\end{aligned}\tag{2.28}$$

The first thing to note is that the LP is feasible if and only if $\mathbf{s} \in K = \{\mathbf{s} : s_1^2 \leq s_2\}$. We can see that a solution of this problem is $x(t) = (t, t^2)$ and $z(\mathbf{x}(t)) = t^2$ since the upper and lower bounds are equal with initial condition $\mathbf{x}(0) = \mathbf{0}$ (so $\dot{\mathbf{x}}(t) = (1, 2t)^T$). Since we know a solution exists, we can attempt to find a solution using the direct method. Applying the first step of RK4, we require the following values: ($\mathbf{x}_0 = \mathbf{x}(0) = \mathbf{0}$)

$$\begin{aligned}\mathbf{k}_1 &= \mathbf{f}(\mathbf{x}_0, z(\mathbf{x}_0)) = (1, 0), \\ \mathbf{k}_2 &= \mathbf{f}(\mathbf{x}_0 + \frac{h}{2}\mathbf{k}_1, z(\mathbf{x}_0)) = (1, h), \\ \mathbf{k}_3 &= \mathbf{f}(\mathbf{x}_0 + \frac{h}{2}\mathbf{k}_2, z(\mathbf{x}_0)) = \left(1, h - \frac{h^2}{4}\right), \\ \mathbf{k}_4 &= \mathbf{f}(\mathbf{x}_0 + h\mathbf{k}_3, z(\mathbf{x}_0)) = \left(1, 2h - h^4 + \frac{h^7}{2} - \frac{h^{10}}{16}\right).\end{aligned}$$

Then we can calculate the next step:

$$\begin{aligned}\mathbf{x}_1 &= \mathbf{x}_0 + \frac{h}{6}(\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4) = \left(h, h^2 - \frac{h^5}{4} + \frac{h^8}{12} - \frac{h^{11}}{96}\right), \\ t_1 &= t_0 + h = h.\end{aligned}$$

In order for our LP to have a feasible solution, we require $s_1^2 = h^2 \leq h^2 - \frac{h^5}{4} + \frac{h^8}{12} - \frac{h^{11}}{96} = s_2$. This is equivalent to when the inequality $-\frac{h^5}{4} + \frac{h^8}{12} - \frac{h^{11}}{96} \geq 0$ holds. We can categorise when this holds by finding the turning points of the left hand side equation. This is when the derivative is equal to zero, so $-\frac{5h^4}{4} + \frac{2h^7}{3} - \frac{11h^{10}}{96} = 0$. We can factorise this as $h^4(-\frac{5}{4} + \frac{2h^3}{3} - \frac{11h^6}{96})$. This has one real root at $h = 0$ and the other roots are all imaginary, so the only real turning point is at $h = 0$. Testing $h = 1$ and $h = -1$ yields negative values, so we realise that $h = 0$ is a local (and global) isolated maximum. Hence $-\frac{h^5}{4} + \frac{h^8}{12} - \frac{h^{11}}{96} \geq 0$ if and only if $h = 0$, so $\mathbf{x}_1 \notin K$ for any suitable choice of h , so our LP is infeasible at our second step and this method fails.

This example was specifically constructed to demonstrate this problem with the direct method, but for many examples (such as our toy model) it does succeed without a numerical instability so we can still use it as a simple baseline method to solve our problem.

2.4 Optimal Basis Algorithm for Solving Standard dFBA Problems

Both of the issues described previously for the direct method (accuracy versus efficiency and numerical instability) can be solved by the algorithm presented by Harwood, Höffner, and

Barton (ibid.). We will present this algorithm now, and show how solving the problem this way gives us much more insight into the system's behaviour.

When simulating a dFBA problem, the first step is to find a solution to our LP problem at our initial time t_0 . We can do this using the simplex method, to obtain an optimal basic solution $\mathbf{v}_0^*$ as well as the corresponding simplex tableau. The only change we have made is instead of calculating the values of the solution directly, as described in Section 2.1, we are obtaining them from our designation of non-basic/basic values and our simplex tableau. This allows us to continue to use the same basis as time progresses.

Consider what happens when we have a small perturbation in our LP problem. It would make sense that if we only have a small change, the chance of our basis remaining optimal is quite high. From Theorem 3, we know that our basic solution will still be optimal after a perturbation if and only if it remains both primal feasible and dual feasible. Since $\boldsymbol{\lambda}_N$ is dependent on $\mathbf{c}$ and Ξ which are unchanged, then the condition of $\boldsymbol{\lambda}_N \geq \mathbf{0}$ or $\boldsymbol{\lambda}_N \leq \mathbf{0}$ remains satisfied. Therefore, to check if our solution remains optimal, we simply need to check that our solution remains primal feasible (that is, $\mathbf{l}_B \leq \mathbf{v}_B \leq \mathbf{u}_B$ holds). We can see how a basis might become primal infeasible in Figure 2.3.

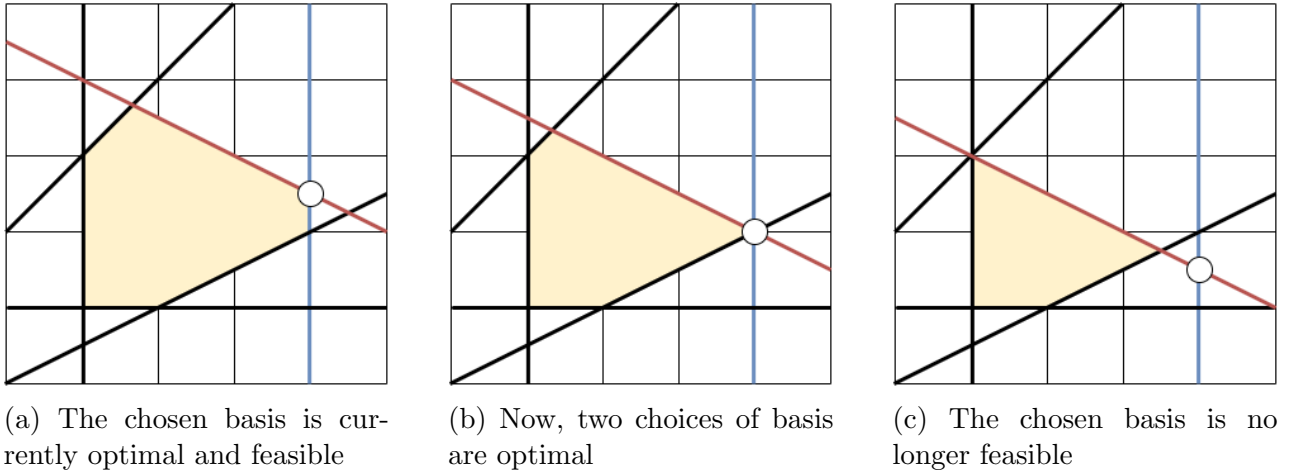


Figure 2.3: An example of how an optimal basis might become infeasible when only the bounds change. The feasible region is represented by the yellow area. The optimal basis is represented by the circle, at the intersection between the red and blue constraints. As we move through the figures, the red constraint becomes tighter, decreasing the size of the feasible region. By the third figure, the basis has moved out of the feasible region, so it is no longer a solution to the respective LP problem.

This motivates the Harwood et al. algorithm described in Algorithm 2. In brief, we solve our LP problem and obtain an optimal basis, then integrate over our system of ODEs using the time-dependent non-basic variables to continuously update the basic variables (using the simplex tableau) until the basis is no longer optimal. The basis remains optimal up until one of our LP variables crosses its upper or lower bound, at which point we re-solve the LP problem for a new optimal basis. Furthermore, the solution to the embedded LP problem is now a continuous function of the kinetic variables over the interval where the basis remains primal feasible. Since our LP variables are determined by algebraic equations, we have actually reduced the problem to solving a Differential-Algebraic system of Equations (DAE) for each of our optimal basis choices. This is beneficial since it allows us to exploit pre-existing packages for solving DAE problems.

From a computational perspective, this algorithm will struggle to find the exact point at which the basis is no longer primal feasible due to small numerical inaccuracies in root detecting algorithms. If we obtain a value slightly less than this switch point, we won't be able to find a

new basis since our old basis is still optimal (and our new basis won't be optimal yet). If we obtain a value slightly more than this switch point, how can we be sure that we have not overreached into a numerical instability? To solve this problem, we instead consider a δ -optimal basis which is defined in the same way except we instead have $l_{B,i} - \delta \leq v_{B,i} \leq u_{B,i} + \delta$ for some fixed $\delta > 0$ and each $i \in \{1, 2, \dots, m\}$. Here, $v_{B,i}$ is the i th value of the vector $\mathbf{v}_B$ and similarly for $\mathbf{l}_B$ and $\mathbf{u}_B$. With this choice, we can now control the level at which we extend past the true switch point. In Harwood, Höffner, and Barton (2016), the correctness of this algorithm is justified since the δ -optimal basis is shown to be a better approximation of the true optimal basis as $\delta \rightarrow 0$. An important note about our choice of software for integrating the system of ODEs is that we are using SUNDIALS (SUite of Nonlinear and Differential/ALgebraic Equation Solvers) (Hindmarsh et al. 2005), (Gardner et al. 2022) - a suite of solvers that can integrate continuously and contains root-finding capabilities. This is essential, since we can describe our condition for infeasibility in terms of a function where a root corresponds to this switch point which we are looking for. For upper bounds this will be a function of the form $u_k - v_k + \delta$ and for lower bounds this will be of the form $v_k - l_k + \delta$. This will be important in Chapter 3, when we extend our algorithm for variable objective functions. By forming this δ -optimal basis we can adjust how close we are to the true switch point, thus allowing us to prevent numerical instabilities.

Another small caveat to highlight is whether our solution is unique. In Harwood, Höffner, and Barton (2016), the focus is on a lexicographic LP problem, where we have multiple objective functions to ensure that we have a unique optimal value for each exchange flux (but not necessarily a unique optimal basis). From this we know that our dFBA problem would then have a unique solution. In this work, we are not considering a lexicographic LP problem so we do not necessarily have a unique solution. To work around this, as long as the basis we choose is dependent on a fixed rule we will have a deterministic algorithm and we will always find the same solution. GLPK, by default, will scan potential new bases until it has found four which it deems to be 'good enough' and takes the best option out of these four. Since the way the new bases are scanned is deterministic, our algorithm will always find the same solution provided we take the same basic solution as an initial condition of the simulation.

To demonstrate an implementation of Algorithm 2, we will apply it to the same toy model as before, from Equation (2.27). We will first solve the problem 'by hand', which will allow us to verify that the implementation is correct (for this example). We will restate the problem here:

$$\begin{aligned}
\dot{x}(t) &= f(x(t), z(x(t))) = -\frac{z(x(t))}{z(x_0)}x(t), \\
x(t_0) &= 1, \\
\text{where } z(s) &= \max_{\mathbf{v} \in \mathbb{R}^3} 10v_1 + 6v_2 + 4v_3, \\
\text{subject to } &v_1 + v_2 + v_3 \leq 100, \\
&10v_1 + 4v_2 + 5v_3 \leq 600s, \\
&2v_2 + 2v_3 + 6v_3 \leq 300, \\
&v_1, v_2, v_3 \geq 0.
\end{aligned} \tag{2.29}$$

We consider this problem over the interval $[t_0, t_{max}] = [0, 2]$. The first step to solve this problem is to transform the LP problem into one with structural and auxiliary variables. This new LP problem is given as follows:

Algorithm 2 Harwood et al. Algorithm

Input: Dynamic LP problem and initial basic solution, initial values of kinetic variables $\mathbf{x}_0$, start time t_0 , finish time t_{max}

Output: Numerical approximation of dynamic LP problem over $[t_0, t_{max})$

- 1: Solve the initial LP problem, starting from the initial basic solution, to find optimal solution $\mathbf{v}_0$, optimal basis and corresponding simplex tableau, Ξ ; if the problem is infeasible, break the algorithm
 - 2: $\mathbf{x}_s \leftarrow \mathbf{x}_0$
 - 3: $t_s \leftarrow t_0$
 - 4: **while** $t_s < t_{max}$ **do**
 - 5: Construct a DAE using the system of ODEs and the algebraic equations from calculating basic variables
 - 6: Integrate the DAE continuously, using $t_s, \mathbf{x}_s$ as the initial conditions until a root is found due to a basic variable crossing its upper or lower bound, or until time t_{max} . Store values of kinetic variables $\mathbf{x}$, objective function z , LP variables $\mathbf{v}$ and the final time as t_f
 - 7: Update the bounds of the LP problem using $\mathbf{x}_{t_f}$ (the kinetic variables calculated at time t_f)
 - 8: Resolve the LP problem for a new optimal basis and Ξ ; if infeasible, break the loop
 - 9: $\mathbf{x}_s \leftarrow \mathbf{x}_{t_f}$
 - 10: $t_s \leftarrow t_f$
 - 11: **end while**
 - 12: **return** all values of $\mathbf{v}, z, \mathbf{x}$
-

$$\begin{aligned} z(s) &= \max_{\mathbf{v} \in \mathbb{R}^6} 10v_1 + 6v_2 + 4v_3, \\ \text{subject to } & v_1 + v_2 + v_3 - v_4 = 0, \\ & 10v_1 + 4v_2 + 5v_3 - v_5 = 0, \\ & 2v_2 + 2v_2 + 6v_3 - v_6 = 0, \\ & v_1, v_2, v_3, v_4, v_5, v_6 \geq 0, \\ & v_4 \leq 100, v_5 \leq 600s, v_6 \leq 300. \end{aligned} \tag{2.30}$$

We have already solved our LP problem at t_0 using the simplex method in Section 2.1 to obtain $z(x_0) = 2200/3 \approx 733$ and $\mathbf{v}_0^* = (100/3, 200/3, 0, 100, 600, 200)^T$. Our non-basic variables are v_3, v_4 and v_5 with v_3 at its lower bound and v_4 and v_5 at their upper bounds. Our basic variables are v_1, v_2 and v_6 , with the simplex tableau given as follows:

$$\Xi_1 = \begin{pmatrix} -\frac{1}{6} & -\frac{2}{3} & \frac{1}{6} \\ -\frac{5}{6} & \frac{5}{3} & -\frac{1}{6} \\ 4 & 2 & 0 \end{pmatrix} \tag{2.31}$$

We can check that $\mathbf{v}_B = \Xi_1 \mathbf{v}_N$ as required for our basic solution. We can also check that $\boldsymbol{\lambda}_N = \mathbf{c}_N + \Xi^T \mathbf{c}_B = (-8/3, 10/3, 2/3)^T$ follows our rule of signs. $\lambda_{N,1}$ corresponds to a lower bound in a maximisation problem so we need $\lambda_{N,1} \leq 0$ and, $\lambda_{N,2}$ and $\lambda_{N,3}$ correspond to upper bounds in a maximisation problem so we need $\lambda_{N,2}, \lambda_{N,3} \geq 0$. Since these are satisfied, we can be sure that this is an optimal solution using Theorem 3.

Our next step is to find when this optimal solution is no longer valid. To do this, we need to check for the first time that any of our basic variables passes over one of its bounds. These are the following options:

$$\begin{aligned}
v_1 &= -\frac{1}{6}v_3 - \frac{2}{3}v_4 + \frac{1}{6}v_5 = 0, \\
v_2 &= -\frac{5}{6}v_3 + \frac{5}{3}v_4 - \frac{1}{6}v_5 = 0, \\
v_6 &= 4v_3 + 2v_4 = 0, \\
v_6 &= 4v_3 + 2v_4 = 300.
\end{aligned} \tag{2.32}$$

Since $v_3 = 0$ and $v_4 = 100$ are fixed we can simplify our equations. We also notice that $v_6 = 200$ does not change since it is not dependent on a dynamically updating non-basic variable, so we can disregard the latter two conditions. The former two conditions can then be rewritten as:

$$\begin{aligned}
-\frac{200}{3} + 100s &= 0, \\
\frac{500}{3} - 100s &= 0.
\end{aligned} \tag{2.33}$$

The first equation is satisfied for $s = 2/3$ and the second equation is satisfied for $s = 5/3$. Considering our function for our kinetic variable, we realise that $z(x(t)) > 0$ since each of our objective coefficients are positive and our variables are non-negative. We also have $x(0) > 0$, so $\dot{x} < 0$ to start with. In order to have $\dot{x} > 0$ we would require $x(t) < 0$ at some point. However, $x(t) = 0$ is an equilibrium, so we cannot move past this point. Hence, we know that $x(t)$ is a strictly decreasing function with $x(t) > 0$. Hence, x will decrease until it reaches $x = 2/3$, at which point we will need to switch our optimal basis.

We can use our optimal basis to solve our initial value problem, and find what time we should expect this switch to occur. We have:

$$\begin{aligned}
v_1(s) &= -\frac{1}{6}v_3 - \frac{2}{3}v_4 + \frac{1}{6}v_5 = 100s - \frac{200}{3}, \\
v_2(s) &= -\frac{5}{6}v_3 + \frac{5}{3}v_4 - \frac{1}{6}v_5 = \frac{500}{3} - 100s, \\
v_3(s) &= 0.
\end{aligned} \tag{2.34}$$

Putting this into our objective function yields:

$$z(x) = 10v_1 + 6v_2 + 4v_3 = \frac{1000}{3} + 400x. \tag{2.35}$$

So, with our initial condition of $z(x_0) = \frac{2200}{3}$, our ODE becomes:

$$\frac{dx}{dt} = -\frac{3}{2200} \left(\frac{1000}{3} + 400x \right) x = -\frac{5}{11}x - \frac{6}{11}x^2. \tag{2.36}$$

This is a first order separable equation, so we can solve it by re-arranging and integrating each side:

$$\int \frac{11}{5x + 6x^2} dx = \int -1 dt = -t + C. \tag{2.37}$$

We solve our left hand side using partial fractions:

$$\begin{aligned}
\int \frac{11}{5x + 6x^2} dx &= \int \frac{11}{5x} - \frac{66}{25 + 30x} dx, \\
&= \frac{11}{5} \ln(x) - 66 \int \frac{1}{25 + 30x} dx, \\
&= \frac{11}{5} \ln(x) - \frac{11}{5} \int \frac{1}{u} du, \text{ where } u = 25 + 30x, \\
&= \frac{11}{5} (\ln(x) - \ln(u)), \\
&= \frac{11}{5} \ln\left(\frac{x}{25 + 30x}\right).
\end{aligned} \tag{2.38}$$

Combining this with our right hand side, we get:

$$\begin{aligned}
\ln\left(\frac{x}{25 + 30x}\right) &= -\frac{5t}{11} + C, \\
\implies \frac{x}{25 + 30x} &= Ce^{-\frac{5t}{11}}, \\
\implies x &= \frac{25Ce^{-\frac{5t}{11}}}{1 - 30Ce^{-\frac{5t}{11}}}.
\end{aligned} \tag{2.39}$$

We can use our initial condition of $x(0) = 1$ to find the value of the constant $C = \frac{1}{55}$ ⁴. So, our solution is given as:

$$x(t) = \frac{\frac{25}{55}e^{-\frac{5t}{11}}}{1 - \frac{30}{55}e^{-\frac{5t}{11}}} = \frac{5e^{-\frac{5t}{11}}}{11 - 6e^{-\frac{5t}{11}}}. \tag{2.40}$$

Finally, we want the value of t when $s = 2/3$, which we can calculate as $-\frac{11}{5} \ln\left(\frac{22}{27}\right) \approx 0.4505s$.

Once $s = 2/3$, we will want to switch to a new basis and we have several choices for this basis. Our variables are equal to $\mathbf{v}^* = (0, 100, 0, 100, 400, 200)^T$, and we have objective value $z = 600$. Since v_1^* and v_2^* are at their lower bounds, and v_4^* and v_5^* are at their upper bounds, any three of these four variables can be chosen as non-basic variables and the remaining three will be basic variables. This demonstrates clearly why we need to overextend slightly.

If instead we take $s = 0.66 < 2/3$ and solve using the simplex method, we find $\mathbf{v}^* = (0, 99, 0, 99, 394, 198)^T$ and $z = 594$. Now, we can clearly see which basis we need to continue to use, namely we have v_2, v_4 and v_6 as our basic variables. From the simplex method, we have our new tableau:

$$\Xi_2 = \begin{pmatrix} -\frac{5}{2} & -\frac{5}{4} & \frac{1}{4} \\ -\frac{3}{2} & -\frac{1}{4} & \frac{1}{4} \\ -3 & \frac{7}{2} & \frac{1}{2} \end{pmatrix} \tag{2.41}$$

Again, we have two non-basic variables which don't have dynamic bounds, v_1 and v_3 , so we have simple equations for when our basis will switch:

⁴We have been a bit loose with C in the calculations here, so it may not be the same between lines, but from here it is fixed.

$$\begin{aligned}
v_2 &= \frac{1}{4}v_5 = 150s = 0, \\
v_4 &= \frac{1}{4}v_5 = 150s = 0, \\
v_6 &= \frac{1}{2}v_5 = 75s = 0, \\
v_6 &= \frac{1}{2}v_5 = 75s = 600.
\end{aligned} \tag{2.42}$$

We previously showed that $x(t)$ is strictly decreasing and bounded from below at 0, so it is impossible for $s = 0$. It is also impossible for $75s = 600$ since this would require $s = 8$ and our initial value was lower than this. Hence, we will not have another basis change in the long term. As $t \rightarrow \infty$, we will have $\mathbf{v} = (0, 0, 0, 0, 0, 0)^T$.

We can find an analytic solution for our kinetic variable, by substituting our basic solution into our objective function:

$$z(x) = 6v_2 = 150x. \tag{2.43}$$

This means our ODE now becomes:

$$\frac{dx}{dt} = -\frac{3}{2200}900x^2 = -\frac{27}{22}x^2. \tag{2.44}$$

Again this is a first order separable equation, so we re-arrange and integrate each side:

$$\begin{aligned}
\int x^{-2}dx &= \int -\frac{27}{22}dt, \\
\implies -\frac{1}{x} &= -\frac{27t + C}{22}, \\
\implies x &= \frac{22}{27t + C}.
\end{aligned} \tag{2.45}$$

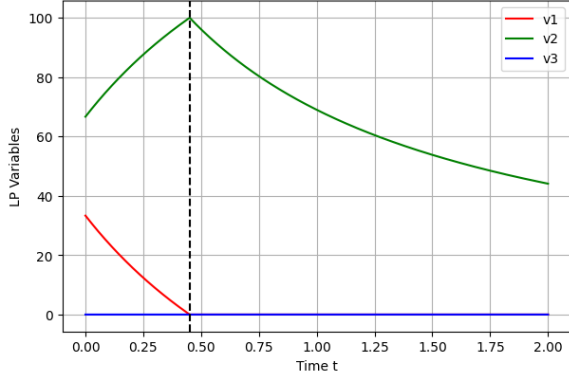
We now have a different initial condition from our previous section of the solution, that being $x(-\frac{11}{5}\ln(\frac{22}{27})) = \frac{2}{3}$. We can plug this into our equation to find $C = 33 + 27(\frac{11}{5}\ln(\frac{22}{27})) \approx 20.8$. We substitute this back to find our solution

$$x(t) = \frac{22}{27t + 33 + 27 * \frac{11}{5} \ln(\frac{22}{27})} = \frac{110}{135t + 165 + 297 \ln(\frac{22}{27})}. \tag{2.46}$$

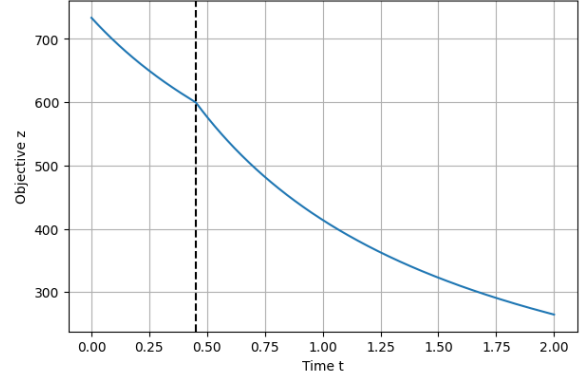
Now we can clearly see that this equation has an asymptote at $x = 0$, so we know that as $t \rightarrow \infty$, $x \rightarrow 0$. Hence, our LP variables will also tend to 0 in the long term. Our problem statement only required us to find a solution for x over the interval of time $[0, 2]$, but we have been able to find a solution that holds for all $t \geq 0$:

$$x(t) = \begin{cases} \frac{5 \exp(-\frac{5t}{11})}{11 - 6 \exp(-\frac{5t}{11})}, & t \in [0, -\frac{11}{5} \ln(\frac{22}{27})] \\ \frac{110}{135t + 165 + 297 \ln(\frac{22}{27})}, & t \in [-\frac{11}{5} \ln(\frac{22}{27}), \infty). \end{cases} \tag{2.47}$$

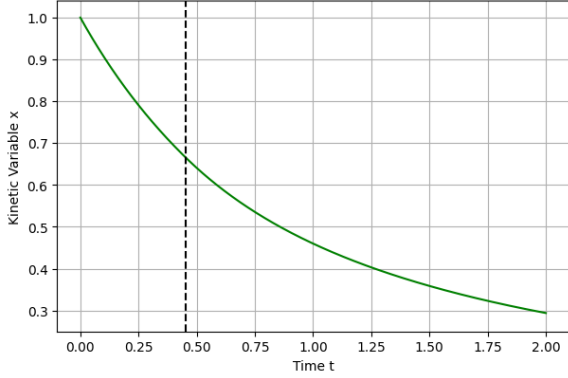
We run Algorithm 2 on this system, and have presented the results in Figure 2.4. The output of the algorithm is the same as that of the direct method (Figure 2.2), which is a promising sign. The difference now is that we also have a deeper understanding of what is happening in the system. In the figures we have marked the time at which the model detects the need to switch basis, which occurs at time $t \approx 0.45$. This is great to see, since our analytical solution had a switch time around $t \approx 0.45$. Our choice of $\delta = 0.00001$ (which decides how far we



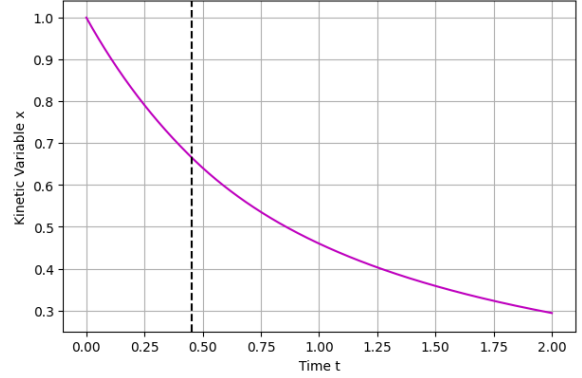
(a) Structural Variables, v_S



(b) Objective value, z



(c) Simulated Solution of x



(d) Analytic Solution of x

Figure 2.4: Solving the toy model (Example 2) using the Harwood et. al method presented in Algorithm 2.

extend past the switch time) is clearly sufficient for an accurate approximation of the switch time, whilst still allowing us to find a new basis.

We also understand that the lack of smoothness in our system is due to this basis change, as we have created a piecewise function with intervals depending on where our bases are optimal. We demonstrate why this change occurs in Figure 2.5.

We can also compare our simulated dynamics to our analytically calculated dynamics. Comparing Figure 2.4c and Figure 2.4d, the two graphs are practically identical and overlaying the two graphs one will see that they overlap completely.

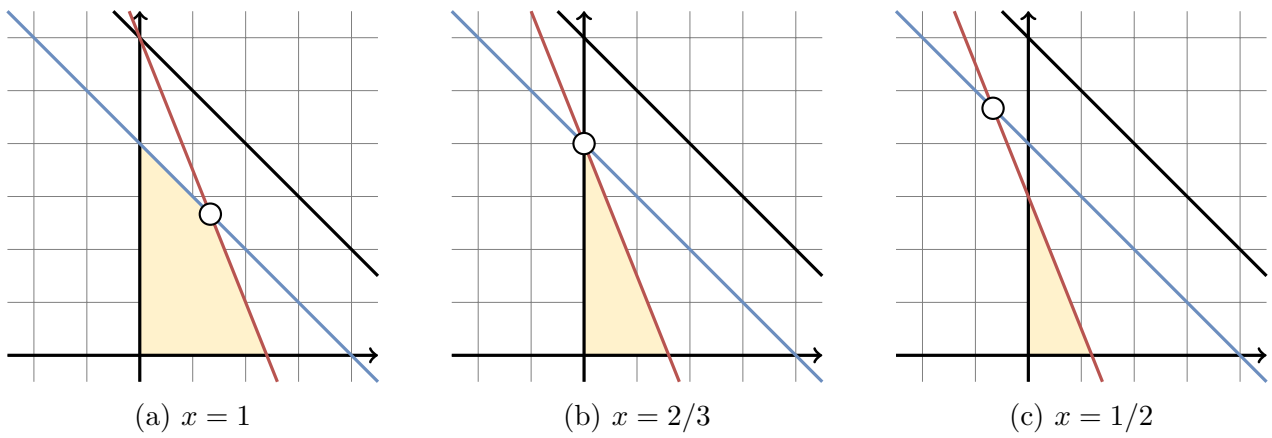


Figure 2.5: Toy model for Example 2 visualised: In this diagram we take a two-dimensional slice of the three-dimensional feasible set. We can do this since v_3 is zero at all times. As the images progress, x decreases and we can see what effect that has on the dynamic bound (in red). This decreases the size of the feasible set (in yellow). The initial choice of basis is shown by the circle, at the intersection of the red and blue constraints. We can see in the first two images that this is feasible (and optimal), but once $x < 2/3$ the point falls out of the feasible set. This means our basis is not primal feasible, so we need to resolve the problem to find a new basis.

Chapter 3

dFBA Problems with Dynamic Objective Functions

Having now seen an efficient algorithm for simulating dFBA problems, that successfully avoids numerical instabilities, we now turn to the main focus of this report. Here our novel contribution will be to consider dFBA with dynamic objective functions and defining and implementing an efficient and robust algorithm for simulating these extended dFBA models.

3.1 Formulation of Extended dFBA Problem

The first step is to introduce the formal problem statement of our extended dFBA models. Many aspects of this statement will be similar to the one previously provided, but we will also need to make adjustments to accommodate the dynamic objective function. Much like in the formal problem statement for standard dFBA models, we will look at the rigorous mathematical framework and then narrow in to the specific biological applications.

Let $D_t \subset \mathbb{R}$, $D_x \subset \mathbb{R}^{n_x}$ and $D_z \subset \mathbb{R}$ be non-empty open sets, representing the domains of time, kinetic variables and the objective function's output, respectively. n_x is the number of kinetic variables in our system, and n_v is the number of LP variables in our system. Let $\mathbf{f} : D_t \times D_x \times D_z \rightarrow \mathbb{R}^{n_x}$, $\mathbf{q} : D_t \times D_x \rightarrow \mathbb{R}^m \times \mathbb{R}^{n_v}$, and $A \in \mathbb{R}^{m \times n_v}$, be given. $\mathbf{f}$ is a function that calculates the value of our kinetic values at a specific time, $\mathbf{q}$ is a function that calculates both the right hand side and objective coefficients of our LP at a specific time (as opposed to just the right hand side) and A is our stoichiometry matrix. For ease of use, we may also use $\mathbf{q}_b$ and $\mathbf{q}_c$ for the functions that calculate the right hand side and objective coefficients respectively. Let $\hat{z} : \mathbb{R}^m \times \mathbb{R}^{n_v} \rightarrow \bar{\mathbb{R}}$ be such that

$$\begin{aligned} \hat{z}(\mathbf{b}, \mathbf{c}) &= \sup_{\mathbf{v} \in \mathbb{R}^{n_v}} \mathbf{c}^T \mathbf{v} \\ &\text{subject to } A\mathbf{v} = \mathbf{b}, \\ &\mathbf{v} \geq \mathbf{0}. \end{aligned} \tag{3.1}$$

Subsequently, define

$$\begin{aligned} F &\equiv \{(\mathbf{b}, \mathbf{c}) \in \mathbb{R}^m \times \mathbb{R}^{n_v} : -\infty < \hat{z}(\mathbf{b}, \mathbf{c}) < +\infty\}, \\ K &\equiv \mathbf{q}^{-1}(F). \end{aligned} \tag{3.2}$$

F represents the set of possible pairs $(\mathbf{b}, \mathbf{c})$ such that the LP is feasible and bounded. $K \subset D_t \times D_x$ is the set of times and kinetic variable values for which the LP is feasible and bounded.

Then, a dFBA problem with a dynamic objective function is formulated as an IVP in ODEs: given an initial time $t_0 \in D_t$ and initial conditions $\mathbf{x}_0 \in D_x$, we seek an interval $[t_0, t_f] = I \subset D_t$, and absolutely continuous function $\mathbf{x} : I \rightarrow D_x$ which satisfy

$$\dot{\mathbf{x}}(t) = \mathbf{f}(t, \mathbf{x}(t), z(t, \mathbf{x}(t))), \quad \text{a.e. } t \in I, \quad \mathbf{x}(t_0) = \mathbf{x}_0, \quad (3.3)$$

where $z : K \rightarrow \mathbb{R} : (t, \mathbf{x}) \mapsto \hat{z}(\mathbf{q}(t, \mathbf{x}))$. Such an I and $\mathbf{x}$ are called a solution of Equation (3.3).

The domain K , which is the set of feasible times and kinetic variables, has been changed to also include the cost coefficients. This is an important distinction, because a problem may become unbounded depending on the choice of objective function. The way we have defined it here allows for many of the theorems that Harwood, Höffner, and Barton (2016) state to have equivalent versions. The first of which is as follows:

Proposition 4 *Assume F defined in Equation (3.2) is non-empty. Then*

$$F = \{(\mathbf{A}\mathbf{v}, \mathbf{c}) : \mathbf{v} \geq \mathbf{0} \text{ and } \exists \mathbf{w} \text{ such that } \mathbf{A}^T \mathbf{w} \leq \mathbf{c}\},$$

and thus it is closed.

other way since we are maximising

Proof. Let $S := \{(\mathbf{A}\mathbf{v}, \mathbf{c}) : \mathbf{v} \geq \mathbf{0} \text{ and } \exists \mathbf{w} \text{ such that } \mathbf{A}^T \mathbf{w} \leq \mathbf{c}\}$. Our goal is to show that $F = S$. First, since F is non-empty we can choose any $(\mathbf{b}, \mathbf{c}) \in F$. It follows that $\hat{z}(\mathbf{b}, \mathbf{c})$ is finite by definition, so the problem is both primal feasible and dual feasible. This means that $\exists \mathbf{v} \geq \mathbf{0}$ such that $\mathbf{A}\mathbf{v} = \mathbf{b}$; and $\exists \mathbf{w}$ such that $\mathbf{A}^T \mathbf{w} \leq \mathbf{c}^1$. So, $F \subseteq S$.

Conversely, since F is non-empty, there exists $(\mathbf{b}^*, \mathbf{c}^*) \in \mathbb{R}^m \times \mathbb{R}^{n_v}$ such that $\hat{z}(\mathbf{b}^*, \mathbf{c}^*)$ is finite. Consequently, $\hat{z}(\mathbf{b}^*, \mathbf{c}^*) = \max\{(\mathbf{b}^*)^T \mathbf{w} : \mathbf{A}^T \mathbf{w} \leq \mathbf{c}^*\}$. The dual problem is feasible for any value of $\mathbf{c}$ such that $\exists \mathbf{w}$ such that $\mathbf{A}^T \mathbf{w} \leq \mathbf{c}$. Thus, using duality results, $\hat{z}(\mathbf{b}, \mathbf{c})$ is finite for all $\mathbf{b}, \mathbf{c}$ such that the primal and dual problems are feasible. These pairs $(\mathbf{b}, \mathbf{c})$ are precisely the elements of S , so $S \subseteq F$.

Combining our two inequalities, we get $F = S$ as required. $\square$

This is an important result, since it allows us to make several more statements about the nature of the problem, which will lead us to our algorithm. These results come from Harwood, Höffner, and Barton (ibid.), and the proofs will be omitted since they are going to be mostly identical. In the following λ denotes the Lebesgue measure.

Theorem 5 *Suppose $\hat{z}^E : \mathbb{R}^m \times \mathbb{R}^{n_v} \rightarrow \mathbb{R}$ is an extension of $\hat{z}$ (this means that $\hat{z}^E$ is defined on all of $\mathbb{R}^m$, and on F , $\hat{z}^E = \hat{z}$), $\mathbf{q}(\cdot, \mathbf{x})$ is measurable for all $\mathbf{x} \in D_x$, $\mathbf{q}(t, \cdot)$ is continuous for almost every $t \in D_t$, and there exists an interval $I^E = [t_0, t_f^E]$ and an absolutely continuous function $\mathbf{x} : I^E \rightarrow D_x$ which are a solution of the IVP*

$$\dot{\mathbf{x}}(t) = \mathbf{f}(t, \mathbf{x}(t), \hat{z}^E(\mathbf{q}(t, \mathbf{x}(t)))), \quad \text{a.e. } t \in I^E, \quad \mathbf{x}(t_0) = \mathbf{x}_0. \quad (3.4)$$

Letting $S(t) = \{s \in [t_0, t] : (s, \mathbf{x}(s)) \notin K\}$, if $(t_0, \mathbf{x}_0) \in K$ and

$$t_f = \sup\{t \in I^E : \lambda(S(t)) = 0\},$$

then $I = [t_0, t_f]$ and $\mathbf{x}$ restricted to I are a solution of the IVP in Equation (3.3). Furthermore, this is the largest interval on which $\mathbf{x}$ is a solution of Equation (3.3).

With this theorem, we know that a solution to our problem exists. It does not help us in finding such a solution though. t_f being dependent on the Lebesgue measure is inconvenient as well, so the following corollary will help us to find the that final time t_f .

¹Appendix A has more information about the dual problem, which is presented here and in the next paragraph.

Corollary 6 Suppose there is a solution $I^E = [t_0, t_f^E]$, $\mathbf{x}$ of the extended ODE in Equation (3.4). Let $S(t) = \{s \in [t_0, t] : (s, \mathbf{x}(s)) \notin K\}$ and $t_f = \sup\{t \in I^E : \lambda(S(t)) = 0\}$. Assume that for any $t \in D_t$, there exists an interval $[t_1, t_2) \subset D_t$ such that $t \in [t_1, t_2)$ and $\mathbf{q}$ is continuous on $[t_1, t_2) \times D_x$. Then $t_f = \inf\{t \in I^E : (t, \mathbf{x}(t)) \notin K\}$.

This corollary allows us to find the exact moment our solution is no longer valid, that is, when $(t, \mathbf{x}(t)) \notin K$.

Much like how Harwood, Höffner, and Barton (2016) showed for their standard dFBA problem, we can prove that our extended dFBA problem has a unique solution. However, as discussed in Section 2.2, this is only true when the only LP variable affecting our kinetic variables is the value of our objective function (as is stated). For dFBA problems, we will often have other fluxes (such as from exchange reactions) affect our kinetic variables, so applying this to a biological example may lead to an issue of uniqueness. The way to solve this issue is to extend the lexicographic method, but this is beyond the scope of this paper.

Our last result will be used to justify the correctness of our algorithm, but we will state it now and return to it later.

locally

Proposition 7 There exists a Lipschitz continuous function $\hat{z}^E : \mathbb{R}^m \times \mathbb{R}^{n_v} \rightarrow \mathbb{R}$ such that $\hat{z}^E$ restricted to F is equal to $\hat{z}$, the solution of the linear program in Equation (3.1).

Lastly, we want to translate this problem so that it is in terms of our structural variables, again so it is similar to how GLPK (Free Software Foundation 2013) is implemented. Because of our choice of domain on z , just like before we can replace the supremum with a maximum. So, we write our system as the following:

$$\dot{\mathbf{x}}(t) = \mathbf{f}(\mathbf{x}(t), z(\mathbf{x}(t))), \quad (3.5)$$

$$\mathbf{x}(t_0) = \mathbf{x}_0, \quad (3.6)$$

$$\text{where } z(\mathbf{s}) = \max_{\mathbf{v} \in \mathbb{R}^n} (\mathbf{0} | \mathbf{c}(\mathbf{s}))^T \mathbf{v}, \quad (3.7)$$

$$\text{subject to } (I | A) \mathbf{v} = 0, \quad (3.8)$$

$$\mathbf{l}(\mathbf{s}) \leq \mathbf{v} \leq \mathbf{u}(\mathbf{s}). \quad (3.9)$$

As to be expected, this is very similar to before, but with the dynamic objective included.

3.2 Extended Optimal Basis Algorithm

We want to extend the Harwood et al. algorithm so that it is capable of reliably simulating dFBA models with dynamic objective functions as well. To demonstrate why the original algorithm is incapable of solving these problems, consider what happens if we have constant bounds and an objective function that updates dynamically - which is allowed by our problem statement. The objective function at any fixed value can be thought of as a hyper-plane with dimension one less than the number of variables. For example, with two variables, our feasible set is a two dimensional area and our objective function is a line. Then, a dynamic update of this objective function relates to some movement of this hyper-plane. This may be a translation or a rotation, which may lead to a different feasible solution to be optimal. An example of this is presented in Figure 3.1.

Suppose we were to try to apply the Harwood et al. algorithm as originally defined to solve this problem. How would our algorithm know when to change basis? The answer is that it would change basis when one of our basic variables breaches its constraints. The basic variables

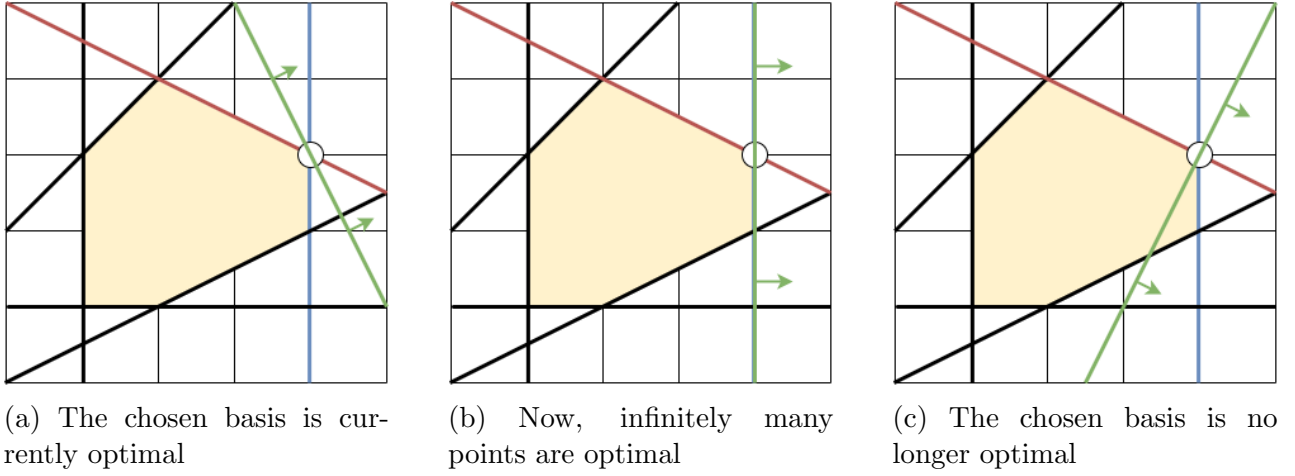


Figure 3.1: An example of how an optimal basis might become infeasible when only the objective function changes. The optimal basis is represented by the circle, at the intersection between the red and blue constraints. The objective function is represented by the green line, with the arrows showing the direction of maximisation. It rotates as we move through the images, which is essentially what happens when our coefficients change.

are dependent on the non-basic variables, which are in turn dependent on the right hand side, which is constant. So, the basic variables will not change and our algorithm will never change basis - which is a problem because we know that there should be a basis change at some point. Therefore, we need to adapt our algorithm somehow.

In order to extend our algorithm, we will return to how we derived the Harwood et al. algorithm. We started by considering what happens when we have a small perturbation in our LP problem. From Theorem 3, we know that our basic solution is still optimal if and only if it is both primal feasible and dual feasible. We already know that we can check primal feasibility by ensuring that no basic variable crosses its lower or upper bounds - and the same holds true here. However, this time we also need to check dual feasibility to ensure our basis remains optimal as the objective function changes. Since our reduced costs $\mathbf{d}$ are dependent on the coefficients in our objective function $\mathbf{c}$, we need to check that the sign of $\mathbf{d}$ remains unchanged when we change $\mathbf{c}$. This is important because we know either $d_i \geq 0$ or $d_i \leq 0$ for each of our reduced costs, so a change in sign suggests a breach of these constraints. Just like before, we will need to overextend slightly to consider a δ -optimal basis, so that we know which basis to choose next. This means when we implement our root-finding functions, they will be of the form $d_i - \delta \leq 0$ or $d_i + \delta \geq 0$ depending on whether that reduced cost relates to an upper or lower bound constraint.

Therefore, our extended algorithm will involve a modest extension of the Harwood et al. algorithm. The extension is that we need to include root-finding functions for each of our reduced costs, and update our cost functions when we re-calculate our basis. We present the algorithm in pseudocode form in Algorithm 3

We want to justify that our algorithm works. To do this, we need to be able to say whether the objective value calculated using a δ -optimal basis will be close to the true objective value at that time and the difference between them will decrease to zero as $\delta \rightarrow 0$.

First, we want to make an important distinction between how the LP problem is constructed in the formal problem statement (Equation (3.1)) and in Section 2.1. Instead of having $A\mathbf{v} = \mathbf{0}$ and $\mathbf{l} \leq \mathbf{v} \leq \mathbf{u}$, we have $A\mathbf{v} = \mathbf{b}$ and $\mathbf{v} \geq \mathbf{0}$. These two definitions are equivalent (see Appendix A), but the equations we derived in Section 2.1 will be slightly different here. The first thing to notice is that since our only bounds are lower bounds at 0, all of our non-basic variables will be equal to 0 and fixed at their lower bound. This means an optimal basis will

Algorithm 3 Extended Algorithm for Variable Right Hand Side and Costs

Input: Dynamic LP problem and initial basic solution, initial values of kinetic variables $\mathbf{x}_0$, start time t_0 , finish time t_{max}

Output: Numerical approximation of dynamic LP problem over $[t_0, t_{max})$

- 1: Solve the initial LP problem, starting from the initial basic solution, to find optimal solution $\mathbf{v}_0$, optimal basis and corresponding simplex tableau, Ξ ; if the problem is infeasible, break the algorithm
 - 2: $\mathbf{x}_s \leftarrow \mathbf{x}_0$
 - 3: $t_s \leftarrow t_0$
 - 4: **while** $t < t_{max}$ **do**
 - 5: Construct a DAE using the system of ODEs and the algebraic equations from calculating basic variables
 - 6: Integrate the DAE continuously, using $t_s, \mathbf{x}_s$ as the initial conditions until a root is found due to a basic variable or a reduced cost crossing its lower or upper bounds, or until t_{max} . Store values of kinetic variables $\mathbf{x}$, objective function z , LP variables $\mathbf{v}$ and the final time as t_f
 - 7: Update the bounds and cost coefficients of the LP problem using $\mathbf{x}_f$ (the kinetic variables calculated at time t_f)
 - 8: Resolve the LP problem again for a new optimal basis and Ξ ; if infeasible, break the loop
 - 9: $\mathbf{x}_s \leftarrow \mathbf{x}_f$
 - 10: $t_s \leftarrow t_f$
 - 11: **end while**
-

have all reduced costs $\mathbf{d} \leq \mathbf{0}$. Also, our equation for calculating our non-basic variables will be slightly different. Instead of having $\mathbf{v}_B = \Xi \mathbf{v}_N$, we will have $\mathbf{v}_B = B^{-1} \mathbf{b}$, and this can be derived in the same way (recall that B is the component of A corresponding to the basic variables, and that Ξ is our simplex tableau). This then allows us to write $\hat{z}^B(\mathbf{b}, \mathbf{c}) = \mathbf{c}^T B^{-1} \mathbf{b}$, and we want to check if this is an accurate approximation of $\hat{z}(\mathbf{b}, \mathbf{c})$ when we are using a δ -optimal basis.

We define $F_B = \{(\mathbf{b}, \mathbf{c}) \in \mathbb{R}^m \times \mathbb{R}^{n_v} : B^{-1} \mathbf{b} \geq \mathbf{0}, \mathbf{c}_N + \Xi^T \mathbf{c}_B \leq \mathbf{0}\}$. The first inequality corresponds to primal feasibility since $B \mathbf{v}_B = \mathbf{b} \implies B^{-1} \mathbf{b} = \mathbf{v}_B \geq \mathbf{0}$. The second inequality corresponds to dual feasibility, as in Theorem 3. This means that F_B is the set of $(\mathbf{b}, \mathbf{c})$ such that our choice of basis is both primal and dual feasible, and thus also optimal. Then, let $F_{B,\delta} = \{(\mathbf{b}, \mathbf{c}) : B^{-1} \mathbf{b} \geq -\delta \mathbf{1}, \mathbf{c}_N + \Xi^T \mathbf{c}_B \leq \delta \mathbf{1}\}$, where $\mathbf{1}$ is a vector of all ones. So, $F_{B,\delta}$ is the set on which our choice of basis is δ -optimal.

We now consider some $(\mathbf{b}, \mathbf{c}) \in F_{B,\delta}$ and let $\mathbf{v} = B^{-1} \mathbf{b}$. We want to construct a solution in an optimal basis that is relatively ‘close’ to our choice of δ -optimal basis². We construct $\tilde{\mathbf{v}}$ and $\tilde{\mathbf{d}}$ such that $\tilde{v}_i = \max\{v_i, 0\}$ and $\tilde{d}_i = \min\{d_i, 0\}$. So $\tilde{\mathbf{v}} \geq \mathbf{0}, \tilde{\mathbf{d}} \leq \mathbf{0}$ and $\|\mathbf{v} - \tilde{\mathbf{v}}\|_\infty \leq \delta$. Then, let $\tilde{\mathbf{b}} = B \tilde{\mathbf{v}}, \tilde{\mathbf{c}}_B = \mathbf{c}_B$ and $\tilde{\mathbf{c}}_N = \tilde{\mathbf{d}} - \Xi^T \tilde{\mathbf{c}}_B$. This choice gives us $(\tilde{\mathbf{b}}, \tilde{\mathbf{c}}) \in F_B$, as well as $\|\mathbf{b} - \tilde{\mathbf{b}}\|_\infty \leq \|B\|_\infty \delta$ and $\|\mathbf{c} - \tilde{\mathbf{c}}\|_\infty = \|\mathbf{d} - \tilde{\mathbf{d}}\|_\infty \leq \delta$. Recall that there exists a Lipschitz continuous function $\hat{z}^E$ which is equal to $\hat{z}$ on F , as described by Theorem 7. So, we have $\hat{z}^B = \hat{z}^E$ on F_B and $\hat{z}^B(\hat{\mathbf{b}}) = \hat{z}^E(\hat{\mathbf{b}})$. Consequently,

²To do this, we require several different norms. For completeness, we will include definitions for each of the norms we present in Appendix C

full stop

$$\begin{aligned}
|\hat{z}^B(\mathbf{b}, \mathbf{c}) - \hat{z}^E(\mathbf{b}, \mathbf{c})| &\leq |\hat{z}^B(\mathbf{b}, \mathbf{c}) - \hat{z}^B(\tilde{\mathbf{b}}, \tilde{\mathbf{c}})| + |\hat{z}^B(\tilde{\mathbf{b}}, \tilde{\mathbf{c}}) - \hat{z}^E(\mathbf{b}, \mathbf{c})| \\
&= |\mathbf{c}_B^T B^{-1} \mathbf{b} - \tilde{\mathbf{c}}_B^T B^{-1} \tilde{\mathbf{b}}| + |\hat{z}^E(\tilde{\mathbf{b}}, \tilde{\mathbf{c}}) - \hat{z}^E(\mathbf{b}, \mathbf{c})| \\
&\leq |\mathbf{c}_B^T B^{-1} \mathbf{b} - \mathbf{c}_B^T B^{-1} \tilde{\mathbf{b}}| + L_z \|(\mathbf{b}, \mathbf{c}) - (\tilde{\mathbf{b}}, \tilde{\mathbf{c}})\|_2 \\
&= \|\mathbf{c}_B^T B^{-1}\|_2 \|\mathbf{b} - \tilde{\mathbf{b}}\|_2 + L_z \|(\mathbf{b}, \mathbf{c}) - (\tilde{\mathbf{b}}, \tilde{\mathbf{c}})\|_2 \\
&\leq M\delta.
\end{aligned}$$

We will explain this line by line. The first line comes from the triangle inequality. In the next line, we use the definitions of $\hat{z}^B$ and $\hat{z}^E$ on the left term, and use the fact that $\hat{z}^B = \hat{z}^E$ on F_B on the right term. The third inequality comes from the Lipschitz continuity of $\hat{z}^E$ and $\mathbf{c}_B = \tilde{\mathbf{c}}_B$. The fourth line uses simple norm properties. For the last line, M is finite, and since we have a finite number of potential bases, we can choose M to be the largest of these choices, so this bound will hold for all bases. However, we also realise this will only hold as long as $\|\mathbf{c}_B^T\|_2$ is bounded from above, which may not be true. To work around this problem, we will only consider cases where the objective function is bounded. This will not be a problematic assumption to make, since we are focusing on multiphase objective functions. This means that we may switch between potential objectives using some dynamic coefficient, but the true numbers of the objective function do not matter - just the ratios between the coefficients.

3.3 Demonstration of Extended Optimal Basis Algorithm on Toy Models

Again, we will demonstrate how the algorithm works using toy models. We will present two simple models to show how the system can complicate very quickly. Our first toy model is a very slight variation on the one presented in the previous chapter (Equation (2.29)):

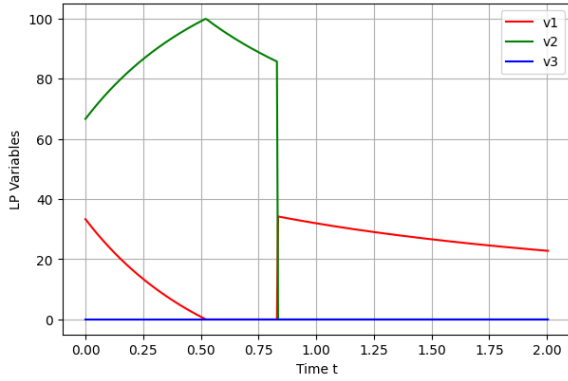
Example 3

$$\begin{aligned}
\dot{x}(t) &= f(x(t), z(x(t))) = -\frac{z(x(t))}{z(x_0)}x(t), \\
x(t_0) &= x_0 = 1, \\
\text{where } z(s) &= \max_{\mathbf{v} \in \mathbb{R}^3} 10v_1 + 7sv_2 + 4v_3, \\
\text{subject to } &v_1 + v_2 + v_3 \leq 100, \\
&10v_1 + 4v_2 + 5v_3 \leq 600s, \\
&2v_1 + 2v_2 + 6v_3 \leq 300, \\
&v_1, v_2, v_3 \geq 0.
\end{aligned} \tag{3.10}$$

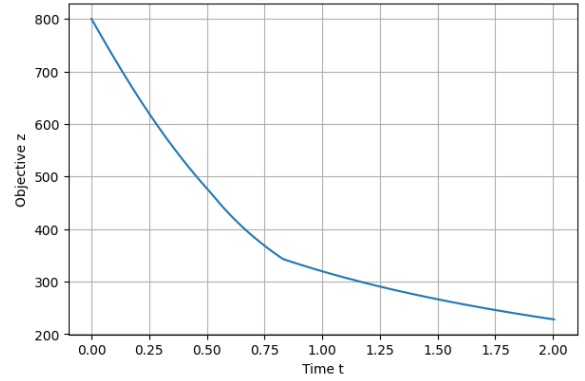
We are considering the time interval $[t_0, t_{max}] = [0, 2]$. First, we check that our cost coefficients are bounded. This is true since we know that $x(t)$ is decreasing and bounded from below by 0, so $0 \leq 7s \leq 7$. Again, we want to rewrite the LP problem in terms of structural and auxiliary variables:

$$\begin{aligned}
z(s) &= \max_{v \in \mathbb{R}^6} 10v_1 + 7sv_2 + 4v_3, \\
\text{subject to } & v_1 + v_2 + v_3 - v_4 = 0, \\
& 10v_1 + 4v_2 + 5v_3 - v_5 = 0, \\
& 2v_2 + 2v_2 + 6v_3 - v_6 = 0, \\
& v_1, v_2, v_3, v_4, v_5, v_6 \geq 0, \\
& v_4 \leq 100, v_5 \leq 600s, v_6 \leq 300.
\end{aligned} \tag{3.11}$$

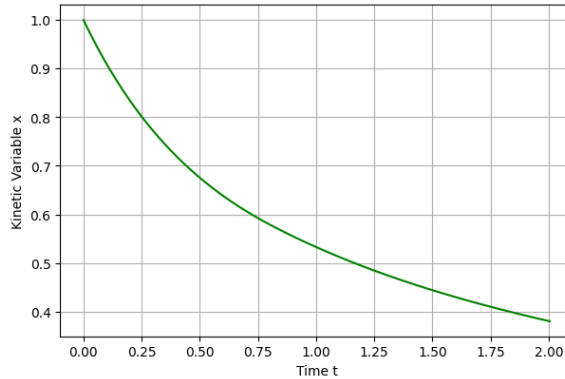
We are now able to apply the direct method to solve this problem. This may be surprising to the reader since the Harwood et al. method does not work, but it makes sense since the direct method is the more naive approach. This means it takes less assumptions, so it does not matter how the dynamics are applied to the LP problem. We can simply update both the bounds and the objective function for every time we resolve the LP problem. The results are presented in Figure 3.2. Analysing these figures, we can see that until time $t \approx 0.8$ the shape of our system looks similar to our previous toy model (Figure 2.2). However, once we pass $t = 0.8$, we get a sharp change in our dynamics. v_2 drops instantly to zero, and v_1 rises to replace it. We also have a ‘less smooth’ shape to our objective function than we did before; there is a much more distinct change in gradient as we hit this point.



(a) Structural Variables, v_S



(b) Objective Function, z



(c) Kinetic Variable x

Figure 3.2: Solving the second toy model (Example 3) using the Direct method.

We will now solve this problem (again by hand and then using Algorithm 3) which may give us insight into why our dynamics are this way.

The only difference in this example and our previous one is that the cost coefficient for v_2 has changed from 6 to $7s$. Because of this, our initial basis is the same with basic variables v_1, v_2 and

v_6 , and $\mathbf{v}^*(t_0) = (100/3, 200/3, 0, 100, 600, 200)^T$, but our initial objective value is now slightly larger with $z(x_0) = 800$. Our simplex tableau also remains unchanged from Equation (2.31), but we now have more conditions on when we change basis. Our reduced costs are given by $\mathbf{d} = \mathbf{c}_N + \Xi^T \mathbf{c}_B$, where:

$$\begin{aligned} d_1 &= 4 - \frac{1}{6} \times 10 - \frac{5}{6} \times 7s + 4 \times 0 = \frac{14 - 35s}{3}, \\ d_2 &= -\frac{2}{3} \times 10 + \frac{5}{3} \times 7s + 2 \times 0 = \frac{35s - 20}{3}, \\ d_3 &= \frac{1}{6} \times 10 - \frac{1}{6} \times 7s + 0 \times 0 = \frac{10 - 7s}{6}. \end{aligned} \quad (3.12)$$

We would need to change basis if $d_i = 0$ for any $i \in \{1, 2, 3\}$. This occurs at $s = \frac{2}{5}, \frac{4}{7}, \frac{10}{7}$ for each equation respectively. Since our last value $\frac{10}{7}$ is greater than one, we can disregard it, as we know that $0 \leq s \leq 1$. We also realise that the other two values are less than $\frac{2}{3}$ which is where our basis would switch before. So, in actuality, the behaviour of our system is the same as before for now, but the numeric values will be different since the solution is not the same.

We can attempt to find an analytic solution to this problem for our first basis using the same techniques as before but we hit a road block once we reach the following equation:

$$\begin{aligned} (13\sqrt{2545} - 509) \ln(-42x + \sqrt{2545} + 65) + 1018 \ln(x) \\ - (509 + 13\sqrt{2545}) \ln(42x + \sqrt{2545} - 65) = \frac{2545t}{3} + C. \end{aligned} \quad (3.13)$$

This equation does not have a solution that can be written in terms of elementary functions, but we can still use it to calculate the necessary values using numerical methods. By applying our initial condition $x(0) = 1$, we can find the value of $C \approx -3227$. Writing the equation in the following form

$$\begin{aligned} g_1(x) &= \frac{3}{2545} ((13\sqrt{2545} - 509) \ln(-42x + \sqrt{2545} + 65) + 1018 \ln(x) \\ &\quad - (509 + 13\sqrt{2545}) \ln(42x + \sqrt{2545} - 65) - C), \end{aligned} \quad (3.14)$$

then allows us to calculate values of x at a given time by calculating $g_1^{-1}(t)$. This is a well-defined function since g_1 is bijective (we will not prove this). We can also find the time at which our switch occurs. This is when $x = 2/3$, so approximately $t = 0.5232$.

We resolve our LP problem to obtain the same second basis as before, with basic variables v_2, v_4 and v_6 and the same simplex tableau as given in Equation (2.41):

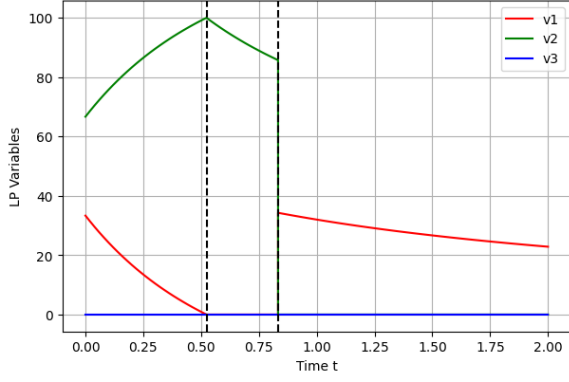
$$\Xi_2 = \begin{pmatrix} -\frac{5}{2} & -\frac{5}{4} & \frac{1}{4} \\ -\frac{3}{2} & -\frac{1}{4} & \frac{1}{4} \\ -3 & \frac{7}{2} & \frac{1}{2} \end{pmatrix} \quad (3.15)$$

We actually can find an analytic expression for the solution this time, since we only have one non-zero structural variable. This is the following solution

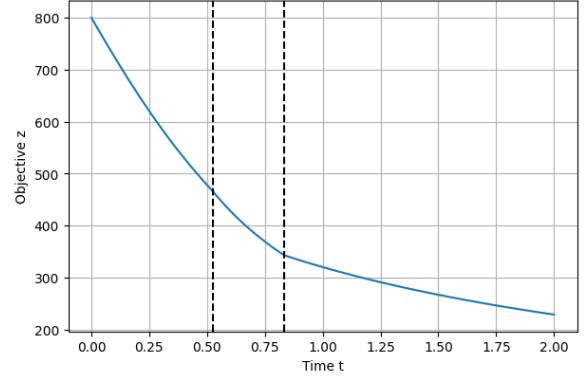
$$x(t) = \sqrt{\frac{8}{21t + C}}. \quad (3.16)$$

We can calculate numerically the value of C ,³ since we know the value of t when $x = 2/3$. This leads us to $C \approx 7.011$.

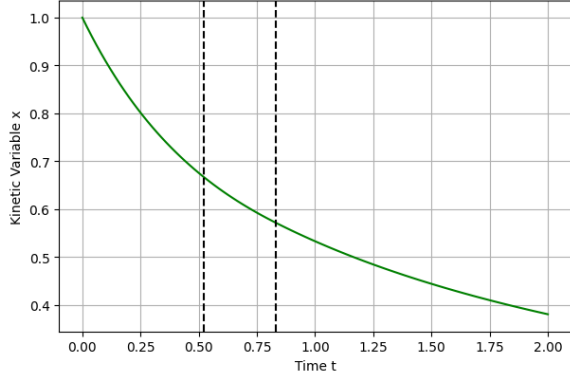
³Again, we are being loose on the notation of C . It is being used as a constant of integration, sometimes for different values in different equations.



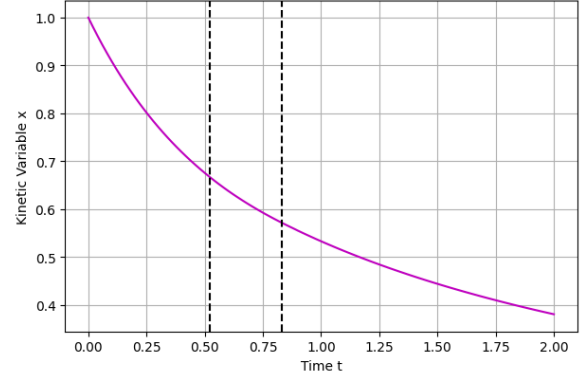
(a) Structural Variables, v_S



(b) Objective Function, z



(c) Kinetic Variable x



(d) Numeric approximation for the 'true' solution of x .

Figure 3.3: Solving the second toy Model 2 (Example 3) using the extended algorithm presented in Algorithm 3.

Previously, we found that our basic variables did not cross their bounds after this point. Since we are using the same basis, this means the basis will not become primal infeasible at any point. This means that if we have another basis change it can only possibly be due to dual infeasibility, i.e. a different basis will have a larger objective function value at some point. To find when this is, we calculate our reduced costs, which are given by:

$$\begin{aligned} d_1 &= 10 - \frac{5}{2} \times 7s - \frac{3}{2} \times 0 - 3 \times 0 = \frac{20 - 35s}{2}, \\ d_2 &= 4 - \frac{5}{4} \times 7s - \frac{1}{4} \times 0 + \frac{7}{2} \times 0 = \frac{16 - 35s}{4}, \\ d_3 &= \frac{1}{4} \times 7s + \frac{1}{4} \times 0 + \frac{1}{2} \times 0 = \frac{7s}{4}. \end{aligned} \tag{3.17}$$

Again, we want to check when our reduced costs become 0. These are at $s = \frac{4}{7}, \frac{16}{35}$ and 0 respectively. We want the largest of these values since x is continuous and will 'reach' that value first. Since $\frac{4}{7} > \frac{16}{35} > 0$, we know that we will have a basis change at $s = \frac{4}{7}$. Using our analytic expression, we find that this is at time $t \approx 0.8328$.

When $s = \frac{4}{7}$ we have two potential optimal bases. The first is the one that is about to become dual infeasible with $\mathbf{v}^* = (0, \frac{600}{7}, 0, \frac{600}{7}, \frac{2400}{7}, \frac{1200}{7})^T$ and $z(\frac{4}{7}) = \frac{2400}{7}$. Our second basis is the one that has just become dual feasible, with $\mathbf{v}^* = (\frac{240}{7}, 0, 0, \frac{240}{7}, \frac{2400}{7}, \frac{480}{7})^T$ and $z(\frac{4}{7}) = \frac{2400}{7}$ again. We can interpret what is happening here is that the strategy of maximising v_2 becomes weaker over time since its cost coefficient is decreasing, so the strategy of maximising the

next best variable v_1 will eventually become dominant. This is different behaviour to when we required a basis switch before. For Example 2, when we switched basis there were many options for basis. Here, we only have two. The difference is that when a solution is about to become dual infeasible, the objective function has become aligned with one of our constraints, so the whole ‘line’ of values become optimal (as in Figure 3.1b). This means the two bases are those at either end of the line. When a solution is about to become primal infeasible, it is because $n_S + 1$ constraints have intersected at the same point (as opposed to the more common n_S) so we have $n_S + 1$ choices of basis, by picking one potential non-basic variable to be excluded from the basis.

Our new simplex tableau is given by:

$$\Xi_3 = \begin{pmatrix} -\frac{2}{5} & -\frac{1}{2} & \frac{1}{10} \\ \frac{3}{5} & \frac{1}{2} & \frac{1}{10} \\ \frac{6}{5} & 5 & \frac{1}{5} \end{pmatrix}. \quad (3.18)$$

For one more time, we want to check our root finding functions. These are given by the following:

$$\begin{aligned} v_1 &= -\frac{2}{5}v_2 - \frac{1}{2}v_3 + \frac{1}{10}v_5 = 60s = 0, \\ v_4 &= \frac{3}{5}v_2 + \frac{1}{2}v_3 + \frac{1}{10}v_5 = 60s = 0, \\ v_4 &= \frac{3}{5}v_2 + \frac{1}{2}v_3 + \frac{1}{10}v_5 = 60s = 100, \\ v_6 &= \frac{6}{5}v_2 + 5v_3 + \frac{1}{5}v_5 = 120s = 0, \\ v_6 &= \frac{6}{5}v_2 + 5v_3 + \frac{1}{5}v_5 = 120s = 300, \\ d_1 &= c_2 - \frac{2}{5}c_1 + \frac{3}{5}c_4 + \frac{6}{5}c_6 = 7s - 4 = 0, \\ d_2 &= c_3 - \frac{1}{2}c_1 + \frac{1}{2}c_4 + 5c_6 = -6 = 0 \\ d_3 &= c_5 + \frac{1}{10}c_1 + \frac{1}{10}c_4 + \frac{1}{5}c_6 = 1 = 0 \end{aligned} \quad (3.19)$$

Since $s \in (0, \frac{4}{7})$, we know that none of these equations can be fulfilled. Therefore, we will not have any more basis changes as $t \rightarrow \infty$. Our long term solution will again have $\mathbf{v} \rightarrow (0, 0, 0, 0, 0, 0)^T$ and $z \rightarrow 0$. Since our only basic structural variables is v_1 , we can find an analytic expression for this section of the solution, using the same technique as before. This yields the following equation

$$x(t) = \frac{4}{3t + C}, \quad (3.20)$$

for some constant of integration C . Since we know the (approximate numeric) value of t when $x = 4/7$, we can calculate $C \approx 4.502$. This means again, our problem will have an asymptote at $x = 0$, and we can verify our long-term behaviour.

Our piecewise ‘true’ solution is given by:

$$x(t) = \begin{cases} g_1^{-1}(t), & t \in [0, t_1), \\ \sqrt{\frac{8}{21t + C_1}}, & t \in [t_1, t_2), \\ \frac{4}{3t + C_2}, & t \in [t_2, \infty), \end{cases} \quad (3.21)$$

where $C_1 \approx 7.011$, $C_2 \approx 4.502$, $t_1 \approx 0.5232$ and $t_2 \approx 0.8328$.

Running our algorithm on the system, we can see the results in Figure 3.3. The dynamics are as expected, v_1 is depleted and v_2 increases over time until we run out of v_1 at around $t = 0.5$. Then, we start to decrease v_2 until we have a sudden switch around $t = 0.8$, where we stop maximising v_2 and start maximising v_1 as this is now the preferable choice. After that, v_1 slowly decreases and will continue to do so as more time passes. We also notice that, unlike our previous algorithm, variables do not have to change continuously when changing basis due to a dual infeasibility. This makes sense, since we are suddenly switching from one ‘strategy’ to another.

We have one more toy model which we will be demonstrating, in order to implement the multiphase dynamic objective. Given that we may use variable objective functions to represent a trade-off between strategies, we want to somehow ‘switch’ strategy depending on the nature of the surrounding environment. Therefore, we can slightly tweak our toy model to get the following:

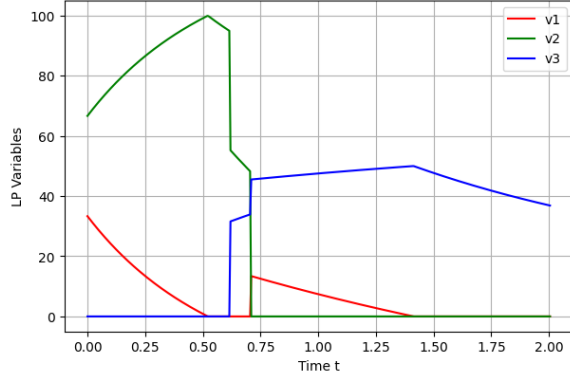
Example 4

$$\begin{aligned} \dot{x}(t) &= f(x(t), z(x(t))) = -\frac{z(x(t))}{z(x_0)}x(t), \\ x(t_0) &= x_0 = 1, \\ \text{where } z(s) &= \max_{v \in \mathbb{R}^3} 10v_1 + 7sv_2 + 15(1-s)v_3, \\ \text{subject to } &v_1 + v_2 + v_3 \leq 100, \\ &10v_1 + 4v_2 + 5v_3 \leq 600s, \\ &2v_2 + 2v_3 \leq 300, \\ &v_1, v_2, v_3 \geq 0. \end{aligned} \tag{3.22}$$

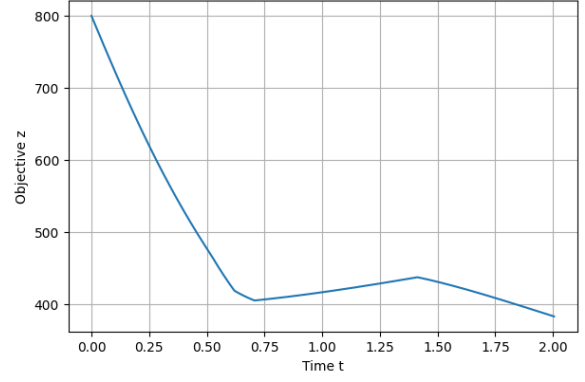
All that has changed here is that our coefficient for v_3 is now $15(1-s)$ instead of 4. This cost function is still bounded, since we know $0 \leq s \leq 1$. Although the objective function coefficients are linear, since we have s decreasing and bounded at 0, the nature of how they progress through time is actually more similar to that of a sigmoid function. This can represent how a certain flux, i.e. v_3 , may become preferable to maximise when the environment changes, i.e. s decreases. The results of the direct method applied to this problem are shown in Figure 3.4.

Using the same techniques to solve the problem by hand, we find our basis changes are when $s = \frac{2}{3}, \frac{12}{19}, \frac{32}{53}, \frac{5}{12} \approx 0.67, 0.63, 0.60, 0.42$. These occur at times around 0.52, 0.62, 0.71 and 1.42 seconds respectively. Because we have so many basis switches, we find that every pair of structural variables is optimal at different points in our time interval.

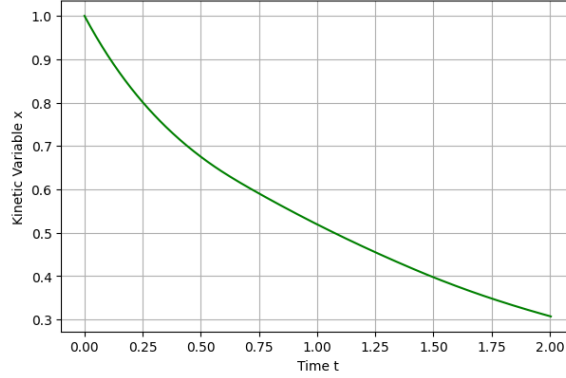
We can make an observation about the nature of our basis changes, depending on whether they are due to primal or dual infeasibility. If we change basis due to primal infeasibility, such as at times $t = 0.52, 1.42$, then our LP variables will change continuously. This makes sense, as in Figure 2.5 we saw that our optimal basis will pass through the basis that we will want to change to before it becomes primal infeasible. Therefore, our LP variables will not experience a sudden jump. Alternatively, our basis may need to change due to dual infeasibility, such as at times $t = 0.62, 0.71$. As we see in Figure 3.1, we see that this is because a new basis which we are switching to can be arbitrarily far from the optimal basis which we are switching from. This means that our LP variables can have massive jumps in their values, and do not need to change continuously. This is seen in Figure 3.4a when the values of our LP variables either change continuously or experience a sharp change. We can interpret the behaviour of the optimal solution as ‘switching strategies’ if we are switching basis due to dual infeasibility, so it will suddenly become preferable to do something completely different.



(a) Structural Variables, v_S



(b) Objective Function, z



(c) Kinetic Variable x

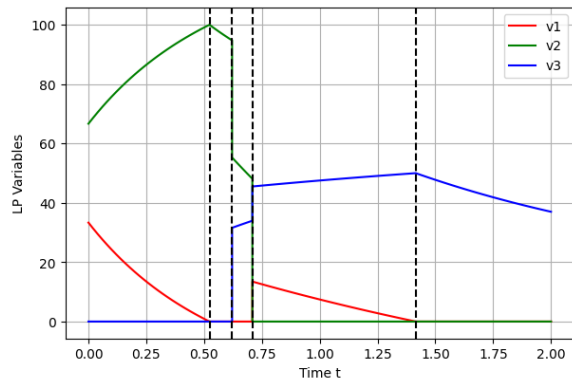
Figure 3.4: Solving the third toy model (Example 4) using the direct method.

Another interesting difference in this version of the toy model is that our objective function is no longer monotonic, it increases at certain points instead of only decreasing. This is interesting, since as our problem becomes more constrained (as in our previous iterations of the toy model) we expect the objective value to decrease. However, now that v_3 becomes more favourable as time passes, it is possible for this increase in value to overcome the loss that the objective experiences due to the problem becoming more constrained.

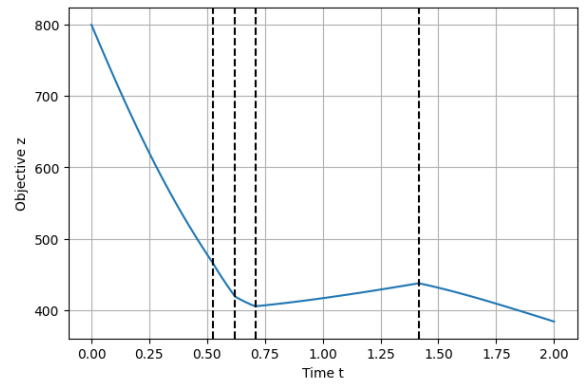
We can calculate numerical approximations of our ‘true’ solution, but for four out of the five different regions, we do not have an analytic expression for this solution. We can write the solution again as a piecewise function:

$$x(t) = \begin{cases} g_1^{-1}(t), & t \in [0, t_1), \\ \sqrt{\frac{8}{21t+C_1}}, & t \in [t_1, t_2), \\ g_2^{-1}(t), & t \in [t_2, t_3), \\ g_3^{-1}(t), & t \in [t_3, t_4), \\ g_4^{-1}(t), & t \in [t_4, \infty), \end{cases} \quad (3.23)$$

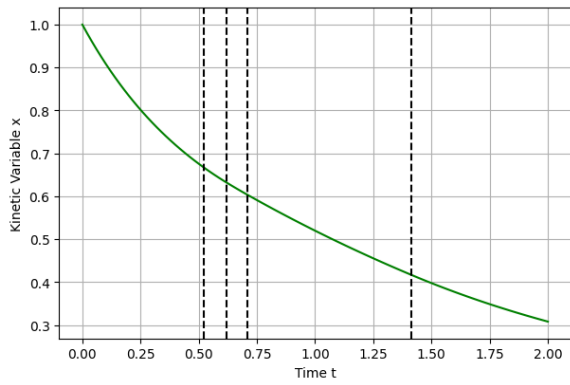
where the functions g_2, g_3 and g_4 are provided in Appendix B. Notably, the equations for the first two sections, including the constant $C_1 \approx 7.011$, are the same as in Example 3. t_1 is the same as before, but $t_2 \approx 0.6212$ is not, since the basis change that would have occurred is instead ‘interrupted’ by v_3 becoming more valuable (previously $t_2 \approx 0.8328$).



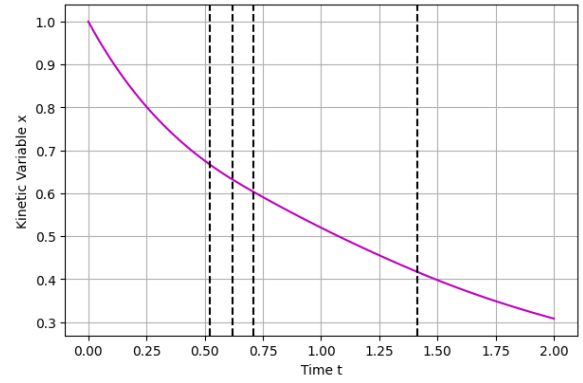
(a) Structural Variables, v_S



(b) Objective Function, z



(c) Kinetic Variable x



(d) Numeric approximation for the 'true' solution of x

Figure 3.5: Solving the third toy model (Example 4) using the extended algorithm.

Chapter 4

Biological Example of an Extended dFBA Problem

4.1 Description of Real World Example

We will now shift our focus to a more biologically motivated example, where it will be slightly more difficult to see the inner workings of the algorithm but we will be much closer to the way that dFBA is used in practice. This example is similar to examples presented in Harwood, Höffner, and Barton (2016), Hanly and Henson (2010) and Gomez, Höffner, and Barton (2014). For the constant objective function version, this is implemented (in a slightly different form) as Example 1 in the software package ‘dfba’ (Tourigny, Muriel, and Beber 2020).

In industry, ethanol is commonly produced using a fermentation process, converting carbon sources such as glucose and xylose into ethanol. In continuous cultures, we can use FBA to model this problem since we will be in (or rapidly reach) a steady-state situation. In a batch culture, however, glucose and xylose in the system will be consumed over time without being continually replaced so our dynamics will change over time. This means we will need to apply dFBA techniques.

We are considering a batch fermentation process using *E. coli* in an anaerobic environment. We start with a large amount of glucose and xylose, which can be consumed by our bacteria and converted into biomass. Since we have two sources of carbon for the bacteria, we should expect to see ‘diauxic growth’. This means that the bacteria will consume the more efficient substrate (in terms of growth rate) until it is practically depleted and only then start consuming the second. Here, glucose is the more efficient option so we should expect xylose consumption to start only when glucose is depleted. In doing this, ethanol is also produced, which is a desirable by-product in industrial applications. We start with an LP problem, which comes from the genome scale metabolic model for *E. coli* (*Escherichia coli str. K-12 substr. MG1655 model* 2003), (Reed, Schilling, and Palsson 2003), using biomass maximisation as our objective function - as is traditional. This model has 761 metabolites and 1075 reactions, which is significantly more than the three of each from our toy model.

Usually, oxygen would be a dynamic variable in this type of problem since it is an important element in metabolism, but since we are working with anaerobic respiration, oxygen concentration will be fixed at zero. So, we have four kinetic variables in our problem: biomass, X ; fuel sources glucose, G , and xylose, Z , and; ethanol, E , as a product. Biomass, as usual, is a representation of the sum of required precursors to make more of a cell. All of these values are concentrations measured in grams per litre (g/L).

The flux through the biomass reaction is given by μ and the fluxes through the exchange reactions are given by v_G , v_Z and v_E , for glucose, xylose and ethanol respectively. These fluxes are measured in grams per gram of dry weight per hour (g/gDW/hr).

Consider how the concentrations of our extracellular metabolites might change over time. The rate at which we consume or produce these metabolites is dependent on how much biomass there is. It also depends on the fluxes through our exchange reactions, so we can write our dynamic equations as:

$$\begin{aligned}\dot{X} &= \mu X, \\ \dot{G} &= -v_G X, \\ \dot{Z} &= -v_Z X, \\ \dot{E} &= v_E X.\end{aligned}\tag{4.1}$$

To construct a dFBA model, the last piece we need is dynamic bounds on our LP problem. One (common) way in which we can do this is by using Michaelis-Menten kinetics to form a lower bound on the rate of consumption. This would lead to the following equations:

$$\begin{aligned}v_{G,\min}(t) &= -v_{G,\max} \frac{G(t)}{K_G + G(t)}, \\ v_{Z,\min}(t) &= -v_{Z,\max} \frac{Z(t)}{K_z + Z(t)} \frac{1}{1 + \frac{G(t)}{K_{iG}}}.\end{aligned}\tag{4.2}$$

where $v_{G,\max}$, $v_{Z,\max}$, K_G , K_z and K_{iG} are constant parameters, which can be obtained from physical experiments. Note that the equation for the xylose flux has an extra term. This is because the presence of glucose has an inhibiting effect on the consumption of xylose - so a large glucose concentration leads to a small lower bound on xylose consumption. This term becomes negligible once glucose becomes small enough.

Finally, we have the following initial conditions: $X(t_0) = 0.03\text{gL}^{-1}$; $G(t_0) = 15.5\text{gL}^{-1}$; $Z(t_0) = 8\text{gL}^{-1}$ and; $E(t_0) = 0\text{gL}^{-1}$.

To finish construction of our dFBA problem, we need to add an objective function. To start with, we will first see what happens when we take the traditional route of considering just biomass maximisation as the objective, $z(t, \mathbf{x}) = \max_{\mathbf{v}} \mu$. This function will have constant coefficients (which is just a 1), so we can apply the direct and Harwood et al. algorithms.

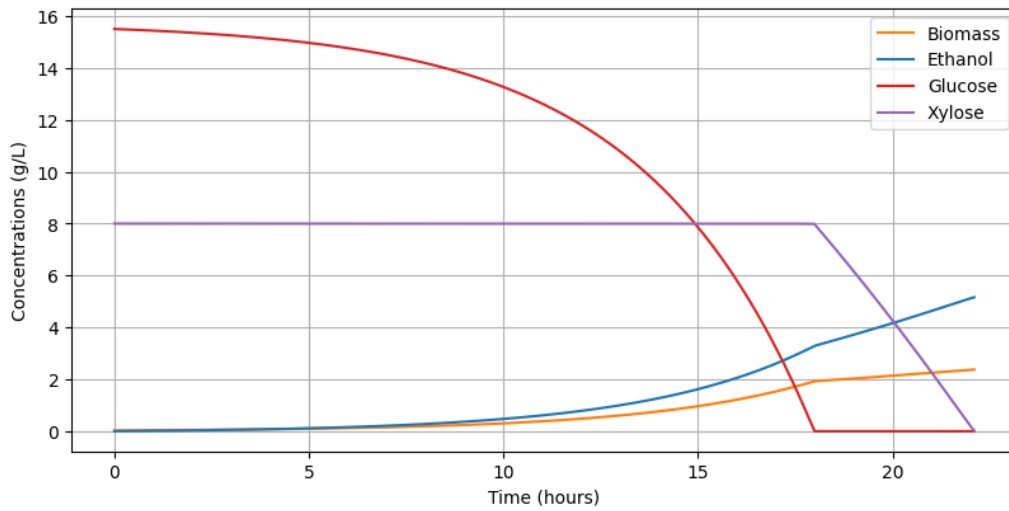
After that, we will present a multiphase objective function

$$z(t, \mathbf{x}(t)) = \max_{\mathbf{v}} \mu - \frac{G(t_0) - G}{100} v_E.$$

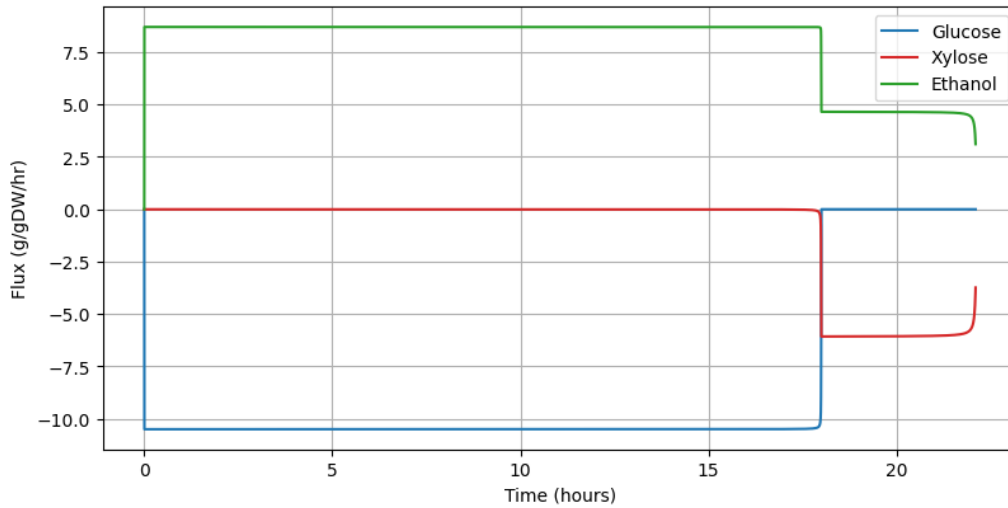
This is representative of how the bacteria may want to focus on different objectives depending on the environment. In this case, our virtual bacteria will want to reduce ethanol production depending on how much glucose is left. Since glucose can only decrease, this coefficient will increase over time. This objective is representative of how an organism may want to protect itself by not polluting its environment. Ethanol is toxic to bacteria in high concentrations (Dyrda, Boniewska-Bernacka, and Man 2019), so as a secondary metabolism technique, the *E. coli* may adapt to prevent itself from causing a high concentration of ethanol. Although this example is not truly biologically rigorous, it is beneficial to demonstrate the ideas behind the desire for this extension. We also need to be wary of the fact that this problem may not have a unique solution, since our exchange fluxes are not necessarily determined uniquely.

4.2 Simulation of Real World Example

We start by modelling the system with the constant objective function using the direct method. The results of this simulation are presented in Figure 4.1, which allows us to verify that the



(a) Kinetic variables: concentrations for biomass, ethanol, glucose and xylose over time (in grams per litre)



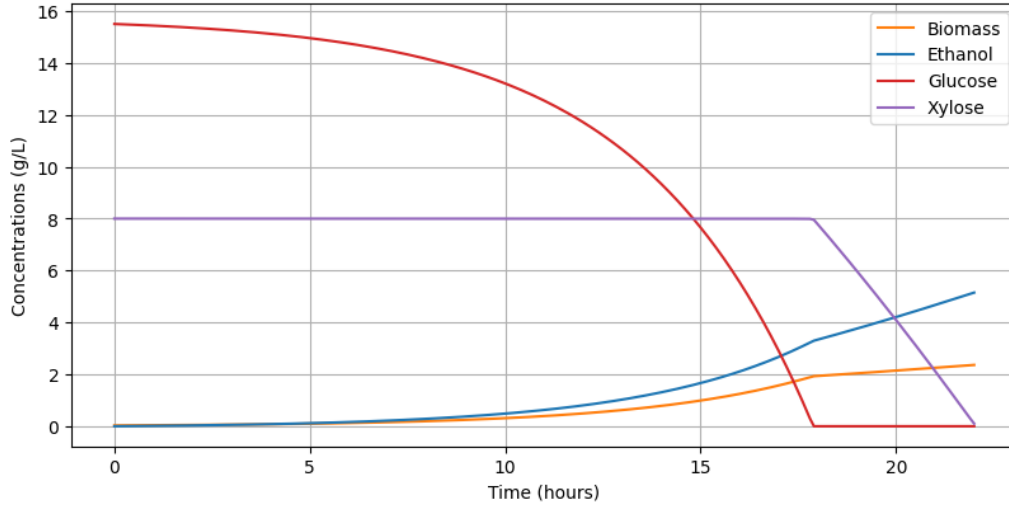
(b) Exchange fluxes for glucose, xylose and ethanol over time (in grams per gram of dry weight per hour)

Figure 4.1: Biological Example: anaerobic fermentation of ethanol through the diauxic growth of *E. coli*, simulated using the direct method (Algorithm 1).

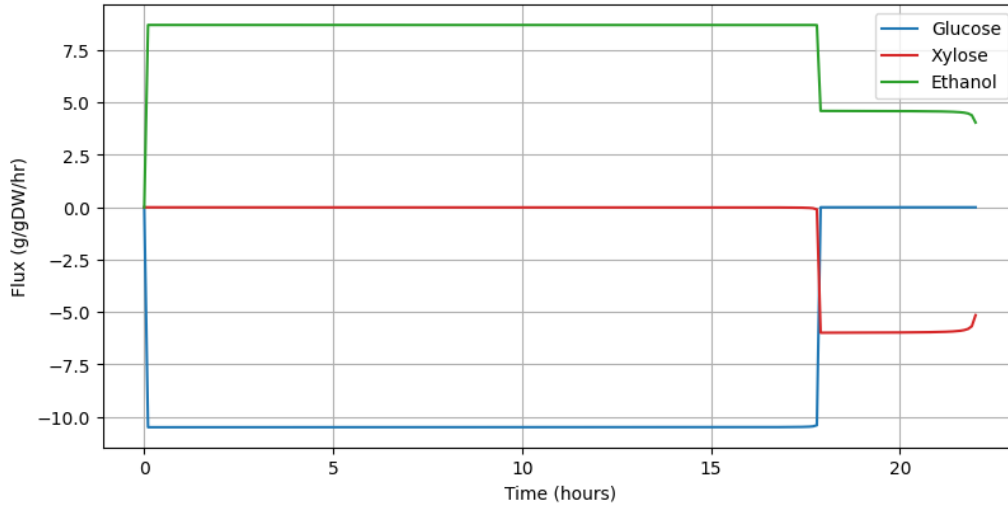
Why not set the colours of the fluxes to be the same as the corresponding concentrations?

behaviour is as expected. We have also applied the Harwood et al. algorithm to the same problem and presented the results in Figure 4.2, but the graphs are identical (even though in this case we do not provide a lexicographic objective). From Figure 4.1a, we can see that the bacteria only consumes glucose for as long as possible, converting it into biomass and producing ethanol as a by-product. This is diauxic growth, as we predicted. This occurs until $t = 18$ hrs, where we have depleted all of the glucose and must switch to consuming xylose instead. Interestingly, the rate at which glucose is consumed accelerates as we increase the amount of biomass we have, whereas the rate of xylose we consume is mostly constant. This may be because biomass increases much slower through consuming xylose than it does through consuming biomass so the change in consumption is negligible. It may also be because xylose consumption is at its upper bound (which is not dynamic) so the xylose supply will decrease at a constant rate. Our system terminates prematurely at $t = 23.9$ hrs since we have consumed all of our xylose and so our LP problem becomes infeasible.

Figure 4.1b demonstrates the values of the fluxes through glucose, xylose and ethanol. As



(a) Kinetic variables: concentrations for biomass, ethanol, glucose and xylose over time (in grams per litre)



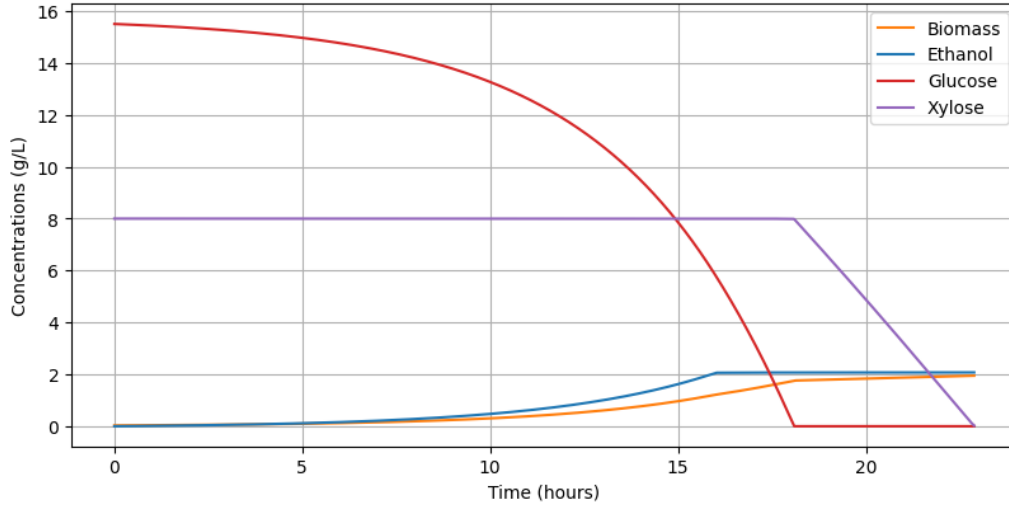
(b) Exchange fluxes for glucose, xylose and ethanol over time (in grams per gram of dry weight per hour)

Figure 4.2: Biological Example: anaerobic fermentation of ethanol through the diauxic growth of *E. coli*, simulated using the Harwood et. al method (Algorithm 2).

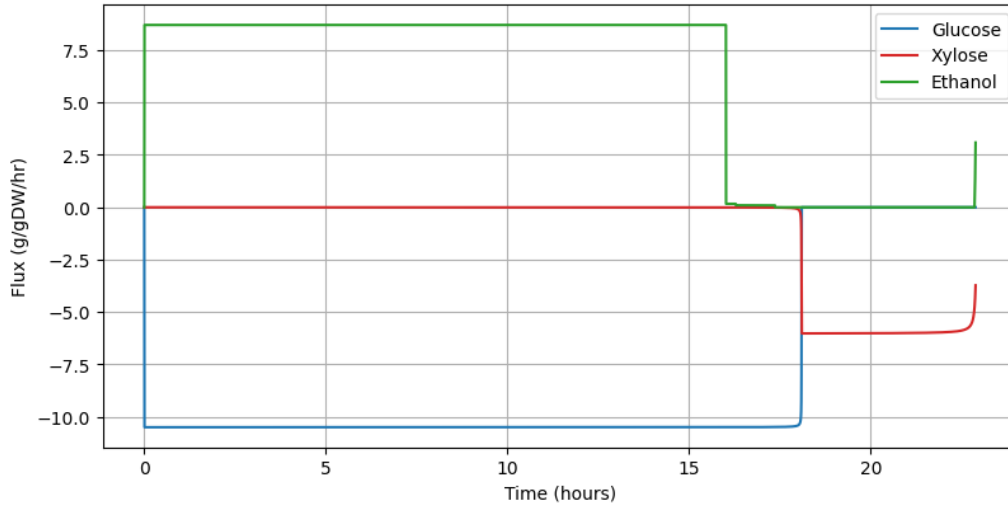
we can see, the flux through xylose starts at zero, with fluxes through ethanol and glucose proportional to each other. Once we deplete all glucose at approximately 18 hours, we have a rapid change in the system. From our understanding of the toy models, this is strange behaviour since the fluxes are meant to change continuously when we have primal infeasibility causing a basis switch. Therefore, we realise that the fluxes are in fact changing rapidly, but not instantly. Glucose consumption becomes zero, xylose consumption begins and ethanol production decreases by a significant amount.

We can check when the basis switches occur, and these are at the following times: 17.8354 hrs, 17.8355 hrs, 17.8399 hrs, 17.8661 hrs, 17.8661 hrs, 17.8796 hrs, 17.8796 hrs, 17.8897 hrs, 17.8897 hrs, 22.0445 hrs. The important thing to notice is that most of these switches occur at practically the same time. We may speculate on what is occurring here. It may be that, due to the large number of variables and constraints, many variables are switching into/out of the basis at the same time but each switch only changes one variable. This means we need a basis switch for every variable that joins/leaves the basis. With this logic, we only have two

unique basis switches. The first is when glucose becomes depleted and we switch to xylose consumption. The second is at the end of the simulation, once the problem becomes infeasible.



(a) Kinetic variables: concentrations for biomass, ethanol, glucose and xylose over time (in grams per litre)



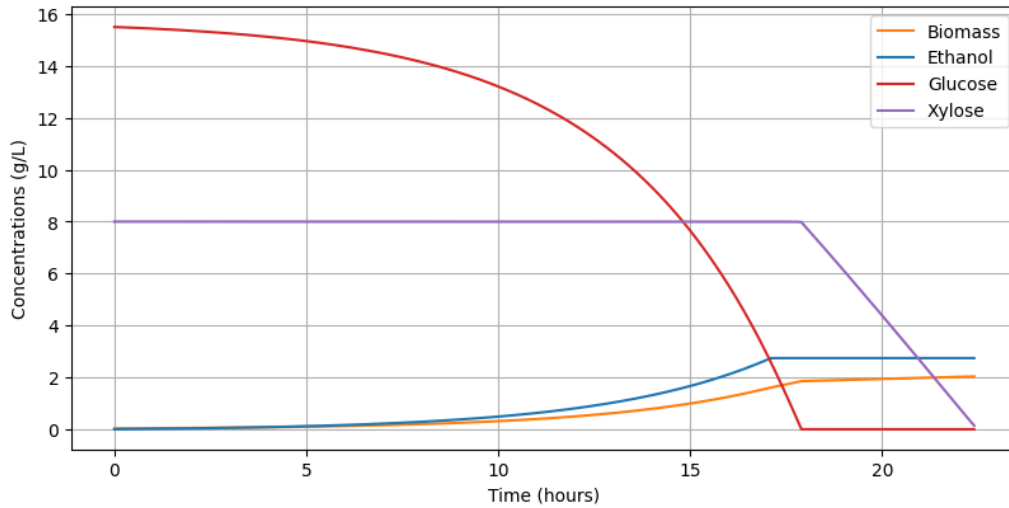
(b) Exchange fluxes for glucose, xylose and ethanol over time (in grams per gram of dry weight per hour)

Figure 4.3: Biological Example: anaerobic fermentation of ethanol through the diauxic growth of *E. coli*, with multiphase objective function, simulated using the direct method.

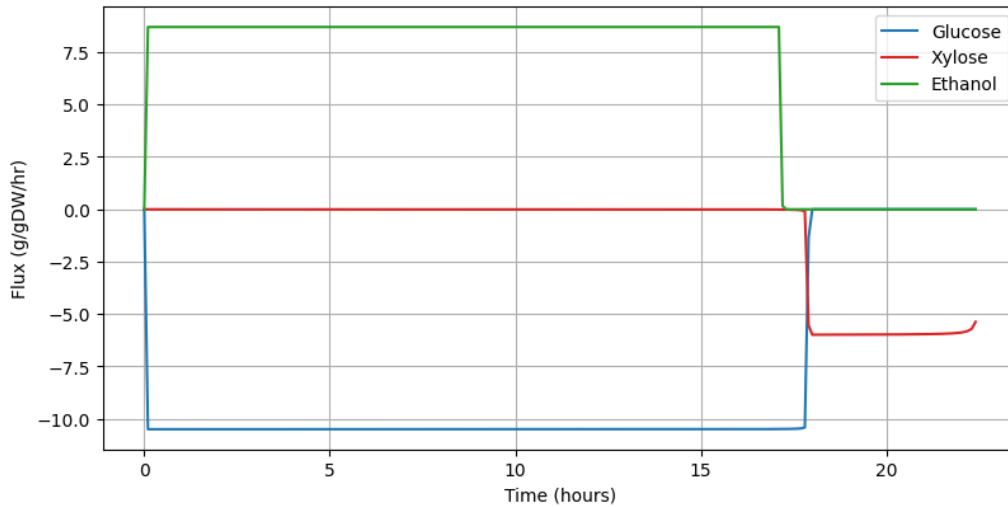
Next, we will apply the direct method to the problem with a multiphase objective function. The results of this are presented in Figure 4.3. Observing Figure 4.3a, what we find in this situation is that our system mostly remains the same as before, up until around 16.0 hours, when glucose concentration has decreased to the point that it is no longer optimal to produce ethanol as a by-product. When it reaches this point, flux through ethanol production reduces to practically zero and ethanol stays fixed at around 2.07gL^{-1} for the rest of the simulation. The rest of the dynamics are the same in that xylose consumption begins once glucose is fully depleted and the simulation ends once xylose is fully depleted as the LP becomes infeasible.

We find that the total biomass produced is significantly reduced compared to the constant objective function case (about 18% lower). This suggests that the optimal production of biomass will always have ethanol as a by-product. This means that reducing ethanol as a method of self-preservation will be at the cost of immediate growth.

We have also run the extended algorithm on the multiphase objective example, with the results presented in Figure 4.4. The results are different from those achieved through the direct method. With the extended algorithm, the bacteria switches to stop producing ethanol at a later time of 17.2 hours. Other than that, the shape of the dynamics is the same as when ran with the direct method. The reason for this disparity is likely due to the non-uniqueness of the problem.



(a) Kinetic variables: concentrations for biomass, ethanol, glucose and xylose over time (in grams per litre)



(b) Exchange fluxes for glucose, xylose and ethanol over time (in grams per gram of dry weight per hour)

Figure 4.4: Biological Example: anaerobic fermentation of ethanol through the diauxic growth of *E. coli*, with multiphase objective function, simulated using the extended algorithm as described in Algorithm 3.

Chapter 5

Conclusions

We have seen traditional models for metabolic metabolism and how they can often be too large or too complicated to solve directly. These models are highly beneficial in industry since microorganisms are vital in the creation of many important products. We showed how in fixed environments we can apply FBA techniques to solve the model as a steady state system. We then justified the dynamic extension of FBA by considering the quasi-steady state that arises from the major disparity between how fast the intracellular and extracellular dynamics occur. We presented an extension of the standard dFBA model to also account for multiphase objective functions, increasing the level of complexity to the problem, which allows us to model more complicated situations. Finally, we have presented a novel algorithm to solve such a model, which is capable of avoiding the numerical instabilities that commonly arise in dFBA simulations. We have also demonstrated these algorithms on many examples, small and large, throughout.

Both the model and the algorithm have many possible directions for further study. This is true from a theoretical perspective and an implementation perspective.

To directly extend this model, the obvious first choice would be to consider a lexicographic LP problem as described in Harwood, Höffner, and Barton (2016). This involves solving the LP problem and, if we have a non-unique solution, we fix the objective value and add it to the set of constraints. We then solve the problem with a new objective, on a smaller feasible set that each have the same initial objective value. We can repeat this process until we have a unique point, or at least when we have a unique value for each of our exchange fluxes as these determine our kinetic variables. This is important from a biological perspective since the system is not just dependent on the objective value (which is always uniquely determined), but also on how the concentrations of metabolites in the system evolve with time. When applying this to a biological example, we would also need to be able to justify why we pick each of our objective functions using experimental data. This type of problem has been studied for a standard dFBA model, but to include a dynamic objective function would be a novel extension.

The benefit of a lexicographic optimisation method is that it ensures uniqueness of our solution to the problem, but another way to solve this issue is to be very specific with our choice of optimal basis. Currently, we are arbitrarily choosing to use the default choice in GLPK, so we do not know if it is a good or bad choice. Alternatively, in Brunner and Chia (2020), a method is described for choosing an optimal basis which will remain optimal for the longest period of time. This is also beneficial from an efficiency perspective, since it ensures that our system will minimise the number of basis changes required.

Another issue with our current work is that we assume the objective function must be bounded to be sure that our algorithm works. We have only focused on systems where the objective coefficients are tightly bounded, but by investigating further we could be more specific on which choice of variable objective functions do or do not lead to an existing solution.

We could also consider more complicated models instead of a more complicated problem. For example, we can consider situations where the state of the system is time dependent such as one where oxygen is supplied only at certain times. We could then combine this with our root finding functions to have complex feed mechanisms. For example, we could continuously supply glucose to the system until it reaches a certain concentration and then cut it off. Doing this we could embed parameters into these feed mechanisms and use many tests to make a recommendation on which choice is optimal.

Our focus in this paper has been on microbial cultures containing just one species. We could also apply the variable objective method to similar problems where we have multiple, interacting species as it is relevant for research on the microbiome. This is described in Henson and Hanly (2014) outside the context of a variable objective function.

Alternatively, we could consider a spatial as well as temporal variation in metabolism, such as when microbial species grow together to form spatial structures known as biofilms. Because of this, these microorganisms do not all have access to the same nutrients required for growth. Those near the surface have rich oxygen supply whereas those buried beneath will only receive what slowly diffuses through. To extend our dFBA models to incorporate such spatial variation, we can allow our kinetic variables to be functions of both space and time, leading to a system of partial differential equations (PDEs) instead, as described in Chen et al. (2016).

Bibliography

- Brunner, J.D. and N. Chia (2020). “Minimizing the number of optimizations for efficient community dynamic flux balance analysis”. In: *PLOS Computational Biology*. DOI: 10.1371/journal.pcbi.1007786.
- Chen, J. et al. (2016). “Spatiotemporal modeling of microbial metabolism”. In: *BMC Syst Biol*. DOI: 10.1186/s12918-016-0259-2.
- Dyrda, G., E. Boniewska-Bernacka, and D. et al. Man (2019). “The effect of organic solvents on selected microorganisms and model liposome membrane”. In: *Molecular Biology Reports* 46. DOI: 10.1007/s11033-019-04782-y.
- Free Software Foundation, Inc. (2013). *GNU Linear Programming Kit, Version 4.52*. URL: <http://www.gnu.org/software/glpk/glpk.html>.
- Gardner, David J et al. (2022). “Enabling new flexibility in the SUNDIALS suite of nonlinear and differential/algebraic equation solvers”. In: *ACM Transactions on Mathematical Software (TOMS)*. DOI: 10.1145/3539801.
- Gomez, J.A., K. Höffner, and P.I. Barton (2014). “DFBAlab: a fast and reliable MATLAB code for dynamic flux balance analysis.” In: *BMC Bioinformatics* 15, p. 409. DOI: 10.1186/s12859-014-0409-8.
- Hanly, T.J. and M.A. Henson (2010). “Dynamic flux balance modeling of microbial co-cultures for efficient batch fermentation of glucose and xylose mixtures”. In: *Biotechnology and Bioengineering*. DOI: 10.1002/bit.22954.
- Harwood, S.M, K. Höffner, and P.I. Barton (2016). “Efficient solution of ordinary differential equations with a parametric lexicographic linear program embedded”. In: *Numer. Math.* DOI: 10.1007/s00211-015-0760-3.
- Henriques, D. et al. (2021). “A Multiphase Multiobjective Dynamic Genome-Scale Model Shows Different Redox Balancing among Yeast Species of the *Saccharomyces* Genus in Fermentation”. In: *mSystems* 6. DOI: 10.1128/mSystems.00260-21.
- Henson, M.A. and T.J. Hanly (2014). “Dynamic flux balance analysis for synthetic microbial communities”. In: *IET Systems Biology Vol. 8 Issue 5*. DOI: 10.1049/iet-syb.2013.0021.
- Hindmarsh, Alan C et al. (2005). “SUNDIALS: Suite of nonlinear and differential/algebraic equation solvers”. In: *ACM Transactions on Mathematical Software (TOMS)* 31.3, pp. 363–396. DOI: 10.1145/1089014.1089020.
- Mahadevan, R., J.S. Edwards, and F.J. 3rd Doyle (2002). “Dynamic flux balance analysis of diauxic growth in *Escherichia coli*”. In: *Biophys J*. DOI: 10.1016/S0006-3495(02)73903-9.
- Meadows, A.L. et al. (2010). “Application of dynamic flux balance analysis to an industrial *Escherichia coli* fermentation”. In: *Metabolic Engineering*. DOI: 10.1016/j.ymben.2009.07.006.
- Moimenta, A.R. et al. (2025). “An integrated multiphase dynamic genome-scale model explains batch fermentations led by species of the *Saccharomyces* genus.” In: *mSystems*. DOI: 10.1128/msystems.01615-24.
- Munn, T. (2025). URL: <https://github.com/thomasmunn0/Masters-Project.git>.
- Murty, Katta G. (1983). *Linear programming*. With a foreword by George B. Dantzig. John Wiley & Sons, Inc., New York, pp. xix+482. ISBN: 0-471-09725-X.

- Reed, J.L., C.H. Schilling, and B.O. Palsson (2003). “An expanded genome-scale model of *Escherichia coli* K-12 (iJR904 GSM/GPR)”. In: *Genome Biol.* DOI: 10.1186/gb-2003-4-9-r54.
- Escherichia coli* str. K-12 substr. MG1655 model (2003). URL: <http://bigg.ucsd.edu/models/iJR904>.
- The Economic Cell Collective (2025). *Economic Principles in Cell Biology*. No commercial publisher — Online open access book. DOI: <https://doi.org/10.5281/zenodo.14604653>.
- Tourigny, D., J.C. Muriel, and M.E. Beber (2020). “dfba: Software for efficient simulation of dynamic flux-balance analysis models in Python”. In: *Journal of Open Source Software*. DOI: 10.21105/joss.02342.
- Waldherr, S., D.A. Oyarzún, and A. Bockmayr (2014). “Dynamic optimization of metabolic networks coupled with gene expression”. In: *Journal of Theoretical Biology* 365. DOI: 10.1016/j.jtbi.2014.10.035.

Appendix A

Further LP Theory

A.1 Transforming LP Problems

Throughout this report we have claimed that LP problems can be written in many different forms. Those which we have shown are the following:

$$\begin{aligned} & \max_{\mathbf{v} \in \mathbb{R}^{2n}} \mathbf{c}^T \mathbf{v}, \\ & \text{subject to } A\mathbf{v} \geq \mathbf{b}. \end{aligned} \tag{A.1}$$

$$\begin{aligned} & \max_{\mathbf{v} \in \mathbb{R}^n} \mathbf{c}^T \mathbf{v}, \\ & \text{subject to } A\mathbf{v} \geq \mathbf{b}, \\ & \mathbf{v} \geq \mathbf{0}. \end{aligned} \tag{A.2}$$

$$\begin{aligned} & \max_{\mathbf{v} \in \mathbb{R}^n} (\mathbf{0} | \mathbf{c})^T \mathbf{v}, \\ & \text{subject to } (I | -A)\mathbf{v} = \mathbf{0}, \\ & \mathbf{l} \leq \mathbf{v} \leq \mathbf{u}. \end{aligned} \tag{A.3}$$

$$\begin{aligned} & \max_{\mathbf{v} \in \mathbb{R}^{nv}} \mathbf{c}^T \mathbf{v}, \\ & \text{subject to } A\mathbf{v} = \mathbf{b}, \\ & \mathbf{v} \geq \mathbf{0}. \end{aligned} \tag{A.4}$$

We refer to Equation (A.2) as canonical form and Equation (A.4) as standard form. We want to show that any LP problem can be put into these two forms, since this allows us to apply the theory which we have developed.

First, we can always make every LP variable non-negative. Consider an LP variable v_i which is not necessarily non-negative. We will introduce two auxiliary variables v_i^+ and v_i^- such that $v_i = v_i^+ - v_i^-$ with $v_i^+, v_i^- \geq 0$. By applying this substitution for each unconstrained variable, our problem will now have $\mathbf{v} \geq \mathbf{0}$.

We can also remove a non-negativity constraint for an LP variable v_i . To do this, we can add a row to our linearity constraints by taking

$$A' = \begin{pmatrix} A \\ e_i^T \end{pmatrix}, \mathbf{b}' = \begin{pmatrix} \mathbf{b} \\ 0 \end{pmatrix}, \tag{A.5}$$

where e_i is the i th unit vector (it contains all zeros except for a one in the i th position). With this choice, $A'\mathbf{v} \geq \mathbf{b}'$ now contains the non-negativity constraint.

Next, we can change the sign on an inequality linearity constraint. We will denote A_i as the i th row of A . Suppose we have $A_i\mathbf{v} \geq b_i$. Then, we can take

$$A' = \begin{pmatrix} A_1 \\ \vdots \\ A_{i-1} \\ -A_i \\ A_{i+1} \\ \vdots \\ A_m \end{pmatrix}, \mathbf{b}' = \begin{pmatrix} b_1 \\ \vdots \\ b_{i-1} \\ -b_i \\ b_{i+1} \\ \vdots \\ b_m \end{pmatrix}, \quad (\text{A.6})$$

which allows us to have $A'_i\mathbf{v} \leq b_i$. Clearly we can reverse this to obtain the other direction of inequality.

We can also convert an inequality constraint into an equality constraint by adding a slack variable. Consider a constraint $A_i\mathbf{v} \geq b_i$. By adding a slack variable s , we get $A_i\mathbf{v} - s = b_i$, with $s \geq 0$. Incorporating this into our LP variables, we get $\mathbf{v}' = (\mathbf{v}|s)^T$ and $A'_i = (A_i | -1)$. We can also reverse this process in the same way, by converting a constraint $A_i\mathbf{v} = b_i$ into $A_i\mathbf{v} + s \geq b_i$ with $s \geq 0$.

So, we have shown that any linear constraint can be written as an inequality or equality constraint and all LP variables can be made non-negative. The last thing to show is that our objective function can always be written as maximising. Suppose our objective was to minimise $\mathbf{c}^T\mathbf{v}$. Then, by taking $\mathbf{c}' = -\mathbf{c}$, we can also write our objective as maximising $(\mathbf{c}')^T\mathbf{v}$. We can also reverse this process in exactly the same way.

Therefore, every LP problem can be written in canonical or standard form.

A.2 Full Rank

We want to provide a definition for full rank since we have used it several times, and justify the assumption that A is full rank.

Definition 8 *An m by n matrix A has full rank if:*

- *the rows of A are linearly independent when $m < n$,*
- *the columns of A are linearly independent when $m > n$,*
- *A is non-singular when $m = n$.*

In all of our LP problems, we are assuming that we have more rows than columns since we only need to apply LP results to underdetermined systems. This means that for A to have full rank, we require the rows of A to be linearly independent. Suppose our LP problem is in standard form (so we have equality constraints) and A does not have full rank. This means that we have a row A_i which can be written as a linear combination of other rows. Without loss of generality, we can take $i = m$. Then, we have

$$A_m = \sum_{j=1}^{m-1} \lambda_j A_j, \quad (\text{A.7})$$

for some constants λ . For each vector $\mathbf{v}$ that satisfies the first $m - 1$ constraints, we must have

$$A_m \mathbf{v} = \left(\sum_{j=1}^{m-1} \lambda_j A_j \right) \mathbf{v} = \sum_{j=1}^{m-1} \lambda_j A_j \mathbf{v} = \sum_{j=1}^{m-1} \lambda_j b_j. \quad (\text{A.8})$$

This right hand side is a constant, so we have $A_m \mathbf{v} = C$ for all $\mathbf{v}$ which satisfy the first $m - 1$ constraints. Without loss of generality, assume $A_{m,1} = 1$ (this is the element in the m th row and the 1st column). Then, we can substitute

$$v_1 = C - \sum_{j=2}^{n_v} A_{m,j} v'_j \quad (\text{A.9})$$

where $\mathbf{v}' = (v_2, v_3, \dots, v_{n_v})^T$. With this choice, we have removed one variable (i.e. column) and one row from A to form an $(m - 1)$ by $n - 1$ matrix A' . In this way, we still have more columns than rows. Either A' now has full rank or we can repeat this process until we reach a small enough matrix that does (since the number of linearly independent rows hasn't changed). Therefore, we can transform an LP problem where A is not full rank into one where A' is full rank - so this assumption is justified.

A.3 Dual Problems

For every LP problem, there exists a dual problem. Solving this dual problem is equivalent to solving the original (primal) problem, and this notion is useful for understanding our algorithms.

As an example, consider the following LP problem in standard form:

$$\begin{aligned} & \max_{\mathbf{v} \in \mathbb{R}^{n_v}} \mathbf{c}^T \mathbf{v}, \\ & \text{subject to } A \mathbf{v} = \mathbf{b}, \\ & \mathbf{v} \geq \mathbf{0}. \end{aligned} \quad (\text{A.10})$$

The dual LP problem is given by:

$$\begin{aligned} & \min_{\mathbf{w} \in \mathbb{R}^m} \mathbf{b}^T \mathbf{w}, \\ & \text{subject to } A^T \mathbf{w} \leq \mathbf{c}. \end{aligned} \quad (\text{A.11})$$

This fact is used in the proof of Theorem 4.

Many times we have used the term 'dual feasible' and this means that there exists a solution to our dual LP problem. The optimal objective values for the primal problem and dual problem are equal. This means, if we fix this value as z , a primal feasible solution has $\mathbf{c}^T \mathbf{v} \leq z$ and a dual feasible solution has $\mathbf{b}^T \mathbf{w} = \mathbf{c}^T \mathbf{v} \geq z$. Clearly we see why a solution that is primal feasible and dual feasible is optimal, since we will have $\mathbf{c}^T \mathbf{v} = z$.

Appendix B

Extra Functions without Analytic Expressions

In Example 4, we stated that we could write the solution to the IVP as the following:

$$x(t) = \begin{cases} g_1^{-1}(t), & t \in [0, t_1), \\ \sqrt{\frac{8}{21t+C_2}}, & t \in [t_1, t_2), \\ g_2^{-1}(t), & t \in [t_2, t_3), \\ g_3^{-1}(t), & t \in [t_3, t_4), \\ g_4^{-1}(t), & t \in [t_4, \infty). \end{cases} \quad (\text{B.1})$$

We will now present those functions. g_1 is the same as in Example 3:

$$g_1(x) = \frac{3}{2545}((13\sqrt{2545} - 509) \ln(-42x + \sqrt{2545} + 65) + 1018 \ln(x) - (509 + 13\sqrt{2545}) \ln(42x + \sqrt{2545} - 65) - C_1). \quad (\text{B.2})$$

This function is valid on the region $[0, t_1)$ where $t_1 \approx 0.5232$ and the constant of integration is given as $C_1 \approx -3227$.

In the second region, we have an analytic solution:

$$x(t) = \sqrt{\frac{8}{21t + C_2}}, \quad (\text{B.3})$$

which is valid on the region $[t_1, t_2)$ where $t_2 \approx 0.6212$ and the constant of integration is given as $C_2 \approx 7.011$.

In the final three regions we do not have analytic solutions so we need to (numerically) invert the following functions:

$$g_2(x) = \frac{112}{108360}(301 \ln(144x^2 - 155x + 60) - 602 \ln(x) - 62\sqrt{215} \tan^{-1}\left(\frac{288x - 155}{7\sqrt{215}}\right) + C_3), \quad (\text{B.4})$$

$$g_3(x) = \frac{40}{3180}(53 \ln(6x^2 - 9x + 10) - 10 \ln(x) - 6\sqrt{159} \tan^{-1}\left(\sqrt{\frac{3}{53}}(4x - 3)\right) + C_4), \quad (\text{B.5})$$

$$g_4(x) = \frac{4}{9} \left(\frac{1}{x} - \ln(x) + \ln(1-x) - C_5 \right). \quad (\text{B.6})$$

The constants of integration are given by $C_3 \approx -337.5$, $C_4 \approx -108.8$ and $C_5 \approx -0.4475$. These functions are valid on the regions $[t_2, t_3)$, $[t_3, t_4)$ and $[t_4, \infty)$ respectively. These times are given as $t_3 \approx 0.7087$ and $t_4 \approx 1.415$.

Appendix C

Norm Definitions

To prove that our extended algorithm (Algorithm 3) accurately approximated the ‘true’ solution, we required the use of several norms which we will give the definition of here.

Definition 9 For $x \in \mathbb{R}$, $|x|$ is the absolute value of x , and is defined as

$$|x| := \begin{cases} x, & \text{if } x > 0, \\ -x, & \text{if } x < 0. \end{cases}$$

Definition 10 For $\mathbf{x} \in \mathbb{R}^n$ with $\mathbf{x} = (x_1, x_2, \dots, x_n)^T$, $\|\mathbf{x}\|_2$ is the 2-norm (or Euclidean norm) of $\mathbf{x}$, and is defined as

$$\|\mathbf{x}\|_2 := \sqrt{x_1^2 + x_2^2 + \dots + x_n^2}.$$

Definition 11 For $\mathbf{x} \in \mathbb{R}^n$ with $\mathbf{x} = (x_1, x_2, \dots, x_n)^T$, $\|\mathbf{x}\|_\infty$ is the ∞ -norm (or infinity norm) of $\mathbf{x}$, and is defined as

$$\|\mathbf{x}\|_\infty := \max_{i \in \{1, 2, \dots, n\}} |x_i|$$

We will not go into more details about norms here, since it is (mostly) unrelated to the focus of this paper.